

Fine-grained Detection and Analysis of Unknown Encrypted Malicious Traffic from Mixed Noisy Labels

Qianwei Meng, Qingjun Yuan*, Meng Shen, *Member, IEEE*, Siqi Lu, Guangsong Li, Jing Tao, Yong Yu, *Fellow, IEEE*, Yongjuan Wang

Abstract—The efficacy of deep learning-based Network Intrusion Detection Systems (NIDS) is critically constrained by the availability of high-quality labeled data. In real-world environments, datasets often suffer from mixed label noise—consisting of both closed-set and open-set noise—which significantly distorts decision boundaries and leads to critical security misses for previously unknown attacks. Achieving robust classification of known traffic while accurately detecting unknown threats in the presence of such mixed noise remains a key challenge. In this paper, we introduce *Sieve*, a robust framework designed for the fine-grained detection and analysis of unknown encrypted malicious traffic in mixed noise conditions. Sieve consists of three collaborative modules: (i) a noise-resilient label correction module that filters out mixed noise using neighbor consistency metrics and confidence-based subset expansion; (ii) a post-hoc detection module that employs Mahalanobis distance in a purified, compact feature space to identify unknown traffic; and (iii) an unknown traffic labeling module that utilizes semi-supervised clustering to facilitate efficient updates of the dataset. Empirical evaluations across four public datasets demonstrate that Sieve significantly outperforms state-of-the-art methods in both known-class classification and unknown-threat detection. Notably, on the Mal_TLS2023 dataset under 50% noise conditions, Sieve maintains robust performance, with accuracy and F1 scores exceeding 94%. Further theoretical analyses corroborate the superiority of the Sieve framework. The code is available at: <https://github.com/niebiakong/Sieve>.

Index Terms—Malicious traffic detection, noisy label, deep learning, unknown attack.

This work is supported by the National Key Research and Development Program of China, No.2023YFB2705000, and the Natural Science Foundation of Henan, No.252300420992.

Qianwei Meng, Siqi Lu, Guangsong Li and Yongjuan Wang are with the Henan Key Laboratory of Network Cryptography Technology, Information Engineering University, Zhengzhou 450001, China (e-mail: mengqw20@163.com; 080lusiqi@sina.com; lgsok@163.com; pinky-wyj@163.com).

Meng Shen is with the School of Cyberspace Science and Technology, Beijing Institute of Technology, Beijing 100081, China (e-mail: shenmeng@bit.edu.cn).

Jing Tao is with the MoE Key Lab for Intelligent Networks and Network Security, Xi'an Jiaotong University, Xi'an, Shaanxi, China (e-mail: jtao@mail.xjtu.edu.cn).

Yong Yu is with the School of Computer Science, Shaanxi Normal University, Shaanxi 710062, China (e-mail: yuyong@snnu.edu.cn).

Qingjun Yuan is with the MoE Key Lab for Intelligent Networks and Network Security, Xi'an Jiaotong University, as well as the Henan Key Laboratory of Network Cryptography Technology, Information Engineering University and the Key Laboratory of Cyberspace Security, Ministry of Education.

Corresponding author: Qingjun Yuan, gcxyuan@outlook.com, Xi'an Jiaotong University, Xi'an 710049, China.

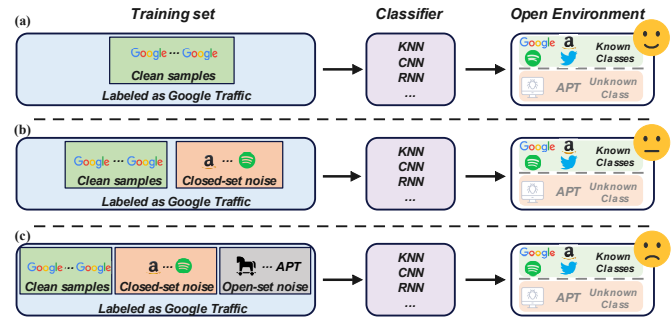


Fig. 1. Detection methods for unknown malicious traffic in three different scenarios: (a) All samples in the training set are clean. (b) Some samples in the training set are incorrectly labeled as other known classes. (c) Some samples of unknown categories are mixed into the training set, and these samples are incorrectly labeled as other known classes.

I. INTRODUCTION

WHILE machine learning and deep learning have become the mainstream solutions for encrypted traffic analysis, their success is heavily predicated on the availability of large-scale, high-quality labeled data [1]–[4]. However, in practical network environments, acquiring pure and accurately labeled datasets faces severe challenges, as the pervasive presence of **label noise**—characterized by mislabeling or inconsistencies—significantly degrades model classification performance and reliability [5].

Label noise in encrypted traffic datasets arises from inherent limitations in automated labeling methods [6] and subjective errors during manual annotation [7]. In practice, this noise typically manifests in a mixed form, comprising both **closed-set noise** (where a sample is mislabeled as another known class) and **open-set noise** (where a sample from an unknown class is incorrectly assigned a known label) [8], [9], as illustrated in Fig. 1. This degradation in label quality severely constrains the practical deployment of malicious traffic classifiers.

These two types of noise harm models differently: closed-set noise leads to blurred decision boundaries between known categories, while open-set noise misleads the model into identifying potential unknown attacks as normal traffic or known malware, causing serious security misses. Furthermore, current research on malicious traffic classification mainly focuses on the closed-world assumption, making it difficult to cope with the endless stream of unknown threats in open environments [10]–[12]. Therefore, achieving precise classification of known

malicious traffic and acute detection of unknown malicious traffic simultaneously under mixed noise conditions is a critical problem.

To address the label noise problem, existing mainstream methods typically employ sample selection strategies [5], [13]–[17], designing screening algorithms to filter noise or correct labels to construct a purer training subset. However, most existing denoising methods ignore the existence of open-set noise and lack targeted filtering mechanisms. On the other hand, to detect unknown malicious traffic, existing studies mostly integrate detection modules into the model training process [18]–[20], which often requires complex model architectures or specific loss function designs. More importantly, in the context of label noise, coupling the detection mechanism directly within the training process makes it highly susceptible to interference from noisy samples, leading to a significant increase in the misclassification rate of unknown traffic.

To overcome the aforementioned challenges, we observe that in the feature space, the label consistency between clean samples and their neighbors is significantly higher than that of noisy samples. Based on this intuition, we propose a robust detection framework named *Sieve*. Sieve consists of three collaborative modules: noisy label correction, unknown traffic detection, and unknown traffic labeling. First, the noisy label correction module utilizes neighbor consistency metrics and prediction confidence to effectively screen and distinguish clean samples, closed-set noise, and open-set noise. It corrects the labels of closed-set noise samples, thereby constructing a pure and class-balanced training set. Second, to avoid noise interference with detection performance, the unknown traffic detection module adopts a “Post-hoc Detection” mechanism. This mechanism identifies unknown traffic using a metric based on the Mahalanobis Distance on top of a model trained on purified data. Finally, addressing the complex composition of detected unknown traffic, the unknown traffic labeling module implements efficient labeling of new attack types and dataset updates by estimating the number of categories and performing cluster analysis, combined with expert intervention.

Sieve is a novel framework for unknown malicious traffic detection that explicitly handles mixed label noise, i.e., the coexistence of closed-set and open-set noise in the training data. In contrast to prior works that consider unknown traffic detection together with closed-set noise handling, Sieve further addresses open-set noise by correcting closed-set noisy labels while rejecting open-set noisy samples. The main contributions of this paper are summarized as follows:

- We propose a novel fine-grained detection framework tailored for low-quality labeled data, named Sieve. This framework exhibits high model compatibility and significantly enhances the encrypted traffic classification and detection capabilities of various supervised learning models within noisy environments.
- We design a noise-resilient feature learning and collaborative detection mechanism. By leveraging innovative sample screening and label correction strategies alongside a decoupled post-hoc detection mechanism, Sieve effectively mitigates mixed noise interference, ensuring

the generation of stable feature representations for the accurate classification of known traffic and the precise capture of unknown threats.

- We verify the comprehensive performance of Sieve across four public datasets. Experimental results demonstrate that Sieve outperforms existing state-of-the-art (SOTA) methods in both known-class classification and unknown-class detection tasks; notably, it exhibits superior robustness in high-noise scenarios, significantly improving the model's accuracy and F1 score.

The rest of this paper is structured as follows: Section II reviews the related literature, and Section III formally defines the problem. Section IV details the design of the proposed Sieve framework, while Section V provides the supporting theoretical justification. The approach is empirically evaluated in Section VI, and Section VIII presents the concluding remarks.

II. RELATED WORK

In this section, we will review some works related to unknown malicious traffic detection and noise label handling. Table I provides a summary of the related works.

A. Unknown Malicious Traffic Detection

FOSS [18] employs the concept of isolation forests by partitioning the data distribution hyperplane to achieve fine-grained detection of unknown attacks. At the same time, the model also has the capability for automatic updates. The core idea of RoNeTC [19] is that classification results for known traffic should have high confidence (low uncertainty). In contrast, unknown traffic should show high uncertainty due to a lack of prior knowledge. RoNeTC dynamically integrates classification decisions from multiple views and evaluates decision uncertainty to classify unknown traffic. TrafficLLM [21] converts raw traffic into rich semantic embeddings and leverages the traffic understanding capabilities of large language models (LLMs) to detect unknown traffic using the classification output head of the LLM. RFG-HELAD [22] utilizes heterogeneous ensemble learning, combining DNNs, contrastive learning, GANs, and deep kNN to detect unknown traffic. Its core concept is based on uncertainty quantification and using generative adversarial networks to learn the distribution of unknown traffic dynamically. In addition, there are some post-hoc out-of-distribution detection algorithms [23]–[25] that are generally model-agnostic and distribution-agnostic, capable of assessing whether a sample belongs to the original distribution during inference with very low computational cost.

FlowLens [26] introduces the Flow Marker Accumulator data structure, which efficiently collects compact flow distribution features in the data plane of programmable switches through quantization and truncation operations, supporting the control plane for machine learning classification. nPrintML [27] designs a unified aligned binary packet representation, nPrint, and combines AutoML to automatically perform feature selection, model search, and hyperparameter tuning, significantly reducing the manual engineering cost of traffic analysis tasks. NetBeacon [28] first deploys a decision tree model

efficiently in the data plane of programmable switches under a multi-stage sequential model architecture, enabling line-speed flow-level feature computation and intelligent traffic analysis. HyperVision [29] constructs a flow interaction graph to model the interaction patterns between flows, using unsupervised graph clustering to detect unknown encrypted malicious traffic in real-time without labeled data.

However, the unknown traffic detection mechanisms in the above works often participate in model training. In the presence of label noise, these detection mechanisms perform poorly and are relatively complex in design [30]–[32].

B. Noisy-Label Handling for Traffic Classification

Current mainstream approaches for handling traffic noise label problems primarily focus on designing label correction algorithms to correct noisy sample labels. FlowRefiner [13] clusters each type of traffic into multiple subclusters, defining a majority label for each subcluster. Labels of samples whose annotations do not align with the majority label are deemed noisy. It then uses maximum confidence to expand the training subset to complete label correction for noisy samples. DSDIR [14] designs a sample relabeling strategy based on confidence and uncertainty metrics, addressing both noise labels and long-tail distribution issues. MCRRe [15] focuses on both label correction and model robust training, designing three different levels of attribute constraints based on traffic distribution in the decision space. RAPIER [16] observes that normal traffic is tightly distributed within a certain region. In contrast, malicious traffic is spread across the feature space, indicating a density difference between normal and malicious traffic, which is used for label noise correction. AEGIS-Net [5] notes the density difference between clean and noisy samples, selecting multiple prototypes for each type of sample to infer the true labels of noisy samples. CoLD [33] analyzes the impact of label noise from a causal relationship perspective, using three components: feature reordering, local joint learning, and causal collaborative denoising, to effectively identify and filter noisy labels.

However, the above works often only consider the existence of closed-set noisy samples, while open-set noise also exists in real scenarios. Existing methods do not adequately handle this, as the true labels of open-set noise are always incorrect, regardless of how label correction is performed [34].

C. Adversarial Noise

Research on adversarial noise focuses on attackers adding carefully designed perturbations to input samples during the inference stage to deceive detection models. For example, Whisper [35] extracts robust features through frequency-domain analysis to resist adversarial packet injections, while Wedjat [36] uses causal networks to model protocol interactions and identify adversarial patterns that violate semantics. The core goal of such work is to improve model robustness against adversarial examples, under the assumption that the training data has accurate labels. Another related line of work, such as tFusion [37], focuses on reducing labeling costs through cross-modal feature fusion and self-supervised

learning, rather than addressing existing label errors. This fundamentally differs from the label noise problem studied in this paper. Label noise originates from inherent errors in the data collection and annotation process, contaminating the supervision signal during training. This paper focuses on how to achieve reliable model training and detection of unknown threats under this more pervasive type of data imperfection.

TABLE I
COMPARISON WITH PRIOR APPROACHES.

Prior Works	Unknown	Closed-set Noise	Open-set Noise	Code
FOSS [18]	✓	✗	✗	✓
RoNeTC [19]	✓	✗	✗	✗
TrafficLLM [21]	✓	✗	✗	✓
RFG-HELAD [22]	✓	✗	✗	✓
HyperVision [29]	✓	✗	✗	✓
AEGIS-Net [5]	✓	✓	✗	✓
FlowRefiner [13]	✗	✓	✗	✗
DSDIR [14]	✗	✓	✗	✓
MCRRe [15]	✗	✓	✗	✓
RAPIER [16]	✓	✓	✗	✓
CoLD [33]	✗	✓	✗	✗
Sieve	✓	✓	✓	✓

Unknown: Unknown Traffic Detection; Closed-set Noise: Closed-set Noise Label Handling; Open-set Noise: Open-set Noise Label Handling.

D. In summary

Current methods typically address noise correction and unknown detection separately. More critically, they rest on different philosophical foundations. Most prior works on label correction operate under a closed-world assumption, aiming to correct all noisy labels into the known set. In contrast, by acknowledging the reality of mixed noise, our philosophy shifts toward differentiating and treating them distinctly: it corrects closed-set noise while actively rejecting open-set noise, thereby purifying the training basis for unknown threat detection.

In the presence of label noise, particularly when open-set noise samples are mixed into the training set, the performance of unknown traffic detection can be significantly affected. How to design appropriate label correction strategies for closed-set noisy samples, as well as screening strategies for open-set noisy samples to minimize the impact of noisy samples on unknown traffic detection, is the problem that this paper aims to address.

III. THREAT MODEL & PROBLEM STATEMENT

A. Threat model

Fig. 2 illustrates the threat model of this work. The NIDS is deployed at the boundary of a trusted internal network, monitoring encrypted traffic passing through the gateway [38]–[40]. Its purpose is to defend against various types of encrypted malicious traffic originating from the untrusted Internet, such as C2 communications, DDoS attacks, and so on. During the NIDS training process (right-hand side), the system is subject

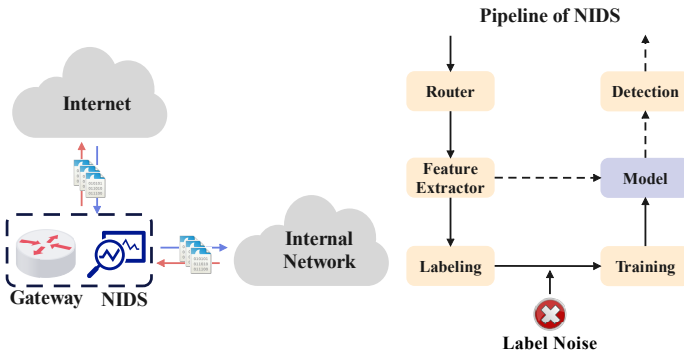


Fig. 2. **Threat model:** The impact of label noise on the NIDS training process. Figure inspired by [33].

to non-adversarial label noise, meaning that the noise arises from labeling errors rather than from deliberate data poisoning by an attacker.

A concrete instance of such harmful open-set noise would be the traffic of a novel, previously unseen ransomware variant being mistakenly labeled as benign during data annotation. This single noisy sample teaches the model to associate the features of this new threat with normal behavior. Consequently, when the actual ransomware attack occurs in deployment, the model will likely misclassify it as benign traffic with high confidence, resulting in a critical security miss.

B. Problem statement

Formally, let $\tilde{V} = \{(x_i, y_i)\}_{i=1}^N$ be a training set with N samples, where $x_i \in \mathbb{R}^d$ is a flow sample and $y_i \in \{0, 1\}^M$ is its one-hot label for M classes. The true label of x_i , denoted as y_i^* , is unobservable. For convenience, we define the logit label $l_i \in \mathcal{C}^l = \{1, 2, \dots, M\}$ as the index where $y_i(j) = 1$. We categorize label noise into two types: **closed-set noise**, where $y_i^* \neq y_i$ but y_i^* still belongs to the known classes, and **open-set noise**, where $y_i^* \neq y_i$ and y_i^* falls outside the M classes.

Using the noisy dataset $\tilde{V} = (\mathcal{X}, \mathcal{Y})$, we aim to train a model parameterized by θ . This model consists of an encoder $\mathcal{F} : x \rightarrow z$ for feature extraction and a classifier $\mathcal{G}_p : z \rightarrow y$ for classification. Additionally, we use a non-parametric KNN classifier $\mathcal{G}_q$ to help select samples. For simplicity, let $\mathcal{F}_i$ be the feature of x_i , p_i be the output of $\mathcal{G}_p$, and q_i be the output of $\mathcal{G}_q$. The presence of mixed noise (both closed-set and open-set) significantly limits the model's ability to classify known traffic and detect unknown threats. The symbols and definitions used in this paper are shown in Table II.

IV. OUR APPROACH

A. Design Philosophy and Overview

Our proposed Sieve framework, as shown in Fig. 3, operates as a robust pipeline designed to mitigate the adverse effects of mixed label noise while enabling the discovery of novel threats. It consists of three integrated modules: *Noisy Label Correction*, *Unknown Traffic Detection*, and *Unknown Traffic Labeling*. Specifically, the *Noisy Label Correction* module employs a neighbor consistency metric and a subset expansion

TABLE II
SYMBOLS AND DEFINITIONS USED IN THIS PAPER.

Notation	Definition
$\tilde{V} = \{(x_i, y_i)\}_{i=1}^N$	Noisy training dataset.
$(\mathcal{X}, \mathcal{Y})$	Samples and labels in $\tilde{V}$.
$(\mathcal{X}_c, \mathcal{Y}_c)$	Selected clean subset, where $c_i \geq \xi$.
$(\mathcal{X}_n, \mathcal{Y}_n)$	Unselected subset: $(\mathcal{X}, \mathcal{Y}) \setminus (\mathcal{X}_c, \mathcal{Y}_c)$.
$(\mathcal{X}_e, \mathcal{Y}_e)$	Expanded subset from $(\mathcal{X}_n, \mathcal{Y}_n)$, where $\max p_i \geq \zeta$.
$(\mathcal{X}_t, \mathcal{Y}_t)$	Final training set: $\text{Rebalancing}((\mathcal{X}_c, \mathcal{Y}_c) \cup (\mathcal{X}_e, \mathcal{Y}_e))$.
$\mathcal{D}_U$	Detected unknown traffic set.
$\mathcal{D}_L$	Labeled reference set, i.e., $(\mathcal{X}_t, \mathcal{Y}_t)$.
$\mathcal{D}_l$	Known-class subset in $\mathcal{D}_U$.
$\mathcal{D}_n$	Novel-class subset in $\mathcal{D}_U$, with $\mathcal{D}_U = \mathcal{D}_l \cup \mathcal{D}_n$ and $\mathcal{D}_l \cap \mathcal{D}_n = \emptyset$.

strategy to meticulously filter clean samples from mixed noise. It specifically corrects mislabeled closed-set samples while discarding open-set noise, thereby constructing a refined and class-balanced training set. Building upon the feature representations learned from this purified data, the *Unknown Traffic Detection* module implements a post-hoc anomaly detection mechanism based on the Mahalanobis distance to accurately identify unknown malicious traffic without interference from noisy samples. Finally, the *Unknown Traffic Labeling* module estimates the number of novel categories within the detected unknown traffic. It performs semi-supervised clustering, facilitating efficient manual annotation by security experts to update the dataset.

The Sieve framework is designed under the philosophy of integrating and optimizing established robust learning paradigms to address the specific challenges of mixed label noise in encrypted traffic. We adapt and build upon several key ideas from prior research: the neighbor consistency principle for sample selection [41], contrastive learning for obtaining compact representations [22], Mahalanobis distance for post-hoc out-of-distribution detection [42] and semi-supervised clustering for exploring unknown categories [43]. However, a straightforward application of these components is insufficient for our target setting. The core of Sieve's contribution lies in how these components are tailored, combined, and augmented to specifically cope with both closed-set and open-set noise while enabling accurate unknown traffic detection.

B. Noisy Label Correction

In the context of a training set that contains both closed-set noise and open-set noise, a natural idea is to select clean samples (i.e., samples whose annotated labels are the correct labels) for training the model. Formally, we treat the entire training set $(\mathcal{X}, \mathcal{Y})$ as the input for a sample selection algorithm, aiming to obtain a clean sample subset $(\mathcal{X}_c, \mathcal{Y}_c)$, where the subscript c denotes clean.

1) *Sample Selection Algorithm:* Our sample selection approach is based on the idea that clean samples often share the same labels as their neighbors [41]. For example, in balanced binary classification, a sample x labeled 0 is considered correct if more than half of its K -nearest neighbors also have the label 0. To quantify this, we use a neighbor consistency metric c_i

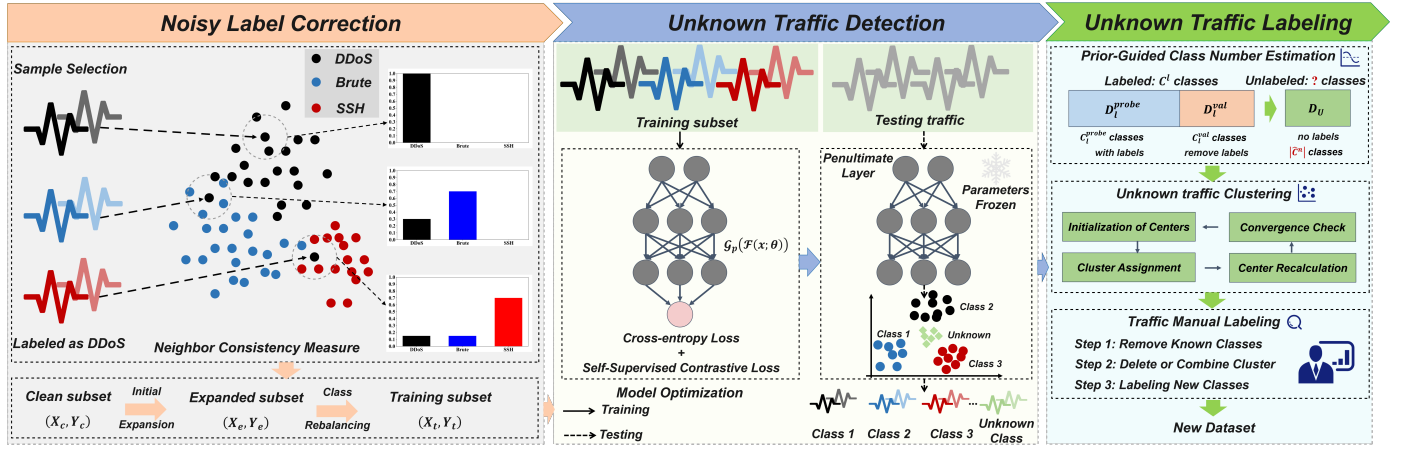


Fig. 3. The architecture of Sieve consists of three components. The noisy label correction module is responsible for filtering out two types of noisy samples and clean samples from the training set, and for correcting the labels of closed-set noisy samples. The unknown traffic detection module builds on the previously trained model to detect unknown malicious traffic. The unknown traffic labeling module first estimates the number of categories in the identified unknown malicious traffic, then performs clustering of the traffic. Finally, experts label the new categories of traffic.

that measures how well the noisy label y_i matches the KNN prediction q_i . Samples with high c_i scores are identified as clean and selected for further training.

Specifically, we first define the similarity s_{ij} between any two samples x_i and x_j , where $\mathcal{F}_i \equiv \mathcal{F}(x_i)$ and we use cosine similarity, leading to $s_{ij} = \cos(\mathcal{F}_i, \mathcal{F}_j)$. Let N_i be the index set of the K most similar samples to x_i based on sample similarity within the set $\mathcal{X}$, with $N_i \subseteq [N]$ and $|N_i| = K$. Then, for each sample x_i , we can calculate the label distribution from KNN voting in its neighborhood, denoted as $q'_i = \frac{1}{K} \sum_{n \in N_i} y_n$. Considering the potential class imbalance in the training set, we first compute the overall label distribution of the dataset as $\chi = \sum_{i=1}^N y_i$, which allows us to derive a balanced label distribution $q_i = \chi^{-1} q'_i$, where χ^{-1} denotes a new vector obtained by taking the reciprocal of each element in χ . In this way, we can mitigate the effects brought by class imbalance.

We let the vector q_i represent the output of the KNN classifier $\mathcal{G}_q$. The neighbor consistency metric c_i is then calculated using the logit label l_i and the KNN prediction q_i as follows:

$$c_i = \frac{q_i(l_i)}{\max_j q_i(j)}, \quad (1)$$

where c_i shows the ratio of the KNN prediction at label l_i to the maximum value across all classes.

We validated this intuition in Fig. 4, where we observed that both closed-set noise and open-set noise exhibited a lower neighbor consistency metric compared to clean samples. A higher neighbor consistency metric c_i indicates that the sample x_i has more consistent labels with its neighboring samples, which means that the annotated label y_i is more likely to be correct, and the sample x_i is more likely to be clean. Based on this, by setting a threshold ξ , the clean sample subset $(\mathcal{X}_c, \mathcal{Y}_c)$ can be obtained through the following filtering strategy:

$$\mathcal{X}_c, \mathcal{Y}_c = \{(x_i, y_i) \mid c_i \geq \xi\}, \quad (2)$$

In the experiments of this paper, we set $\xi = 1$ to obtain a purer sample, meaning that we only consider the sample x_i

as clean if its neighbor's voting label distribution q_i equals its current annotated label y_i .

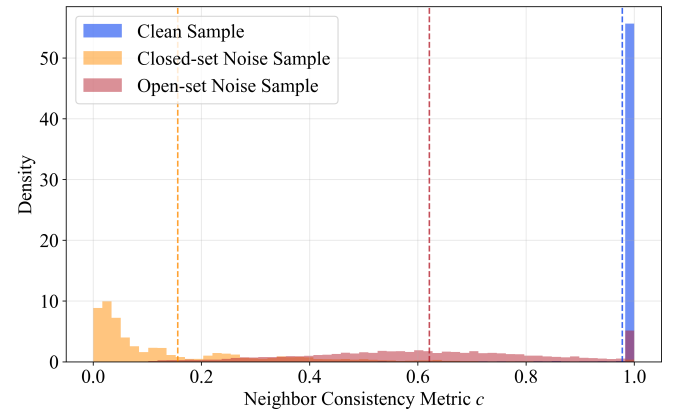


Fig. 4. In the Mal_TLS2023 dataset, under the conditions of 50% symmetric noise rate and 50% open-set noisy samples, the Sieve model was trained for 10 epochs. The distribution of the neighbor consistency metric c scores for clean samples and noisy samples (including closed-set noise and open-set noise) showed that the mean of the neighbor consistency metric for clean samples was 0.9777, the mean for closed-set noise was 0.1561, and the mean for open-set noise was 0.6212.

2) *Subset Expansion*: Relying only on the clean subset $(\mathcal{X}_c, \mathcal{Y}_c)$ is often suboptimal because it ignores a large portion of the data and may still contain residual noise. Furthermore, the model must be robust against class imbalance and various noise patterns. To leverage the unselected samples $(\mathcal{X}_n, \mathcal{Y}_n) = (\mathcal{X}, \mathcal{Y}) \setminus (\mathcal{X}_c, \mathcal{Y}_c)$, we propose a subset expansion mechanism consisting of (a) **initial expansion** and (b) **class rebalancing**.

a) **Initial Expansion**. This step focuses on samples in $\mathcal{X}_n$ where the predicted probability p_i from classifier $\mathcal{G}_p$ exceeds a threshold ζ . These high-confidence samples fall into two groups: (1) samples where the annotated label matches the prediction, likely representing missed clean samples; and (2) samples with label-prediction mismatches, which we consider mislabeled. For the latter, the correct labels are inferred using

the model's predictions. The expanded subset $(\mathcal{X}_e, \mathcal{Y}_e)$ is defined as:

$$(\mathcal{X}_e, \mathcal{Y}_e) = \left\{ (x_i, y'_i) \mid \max p_i \geq \zeta \right\}, \quad (3)$$

where y'_i is the predicted label for sample x_i .

The efficacy of the high-confidence threshold ζ in filtering out open-set noise stems from a fundamental difference between closed-set and open-set noise. For a sample whose true category is outside the known label space $\mathcal{C}^l$ (i.e., open-set noise), the classifier $\mathcal{G}_p$, trained exclusively on known categories, cannot assign a high prediction probability to any single known class. Its predictions tend to be distributed with low confidence across multiple known classes because the sample's feature representation does not align well with any learned class centroid. In contrast, a closed-set noisy sample or a missed clean sample, whose true label resides within $\mathcal{C}^l$, can eventually be recognized by a well-trained model, leading to a high prediction confidence for its true class.

Experimental validation indicates that by setting a high threshold ζ , we can ensure that the initial expanded subset contains samples with high-accuracy labels. Moreover, as illustrated in Fig. 5a, this mechanism can effectively avoid selecting open-set noisy samples, as these typically lack high-confidence predictions and thus have lower $\max p_i$ values. Furthermore, as shown in Fig. 5b, as the number of training epochs increases, the Sieve model becomes increasingly "confident" in its predictions for clean samples; the Clean curve consistently exceeds the Noisy curve. This indicates that the Sieve model can distinguish between clean and noisy samples.

b) Class Rebalancing. By adding the expanded subset $(\mathcal{X}_e, \mathcal{Y}_e)$ to the clean subset $(\mathcal{X}_c, \mathcal{Y}_c)$, we can mitigate the sample imbalance issue to some extent. However, the above measures do not completely resolve the class imbalance problem. The causes of class imbalance are diverse: (1) the initial noisy flow dataset $\tilde{V}$ might be class-imbalanced, and (2) even if $\tilde{V}$ is class-balanced, the subsets obtained after applying the sample selection algorithm and the initial expansion mechanism, $(\mathcal{X}_c, \mathcal{Y}_c)$ and $(\mathcal{X}_e, \mathcal{Y}_e)$, might still be class-imbalanced. For instance, the class A samples in $(\mathcal{X}_c, \mathcal{Y}_c)$ might number 1000, while class B samples could only be 50, or, in extreme cases, there may be no samples from class B in $(\mathcal{X}_c, \mathcal{Y}_c)$ at all.

To fix class imbalance, we apply two post-hoc steps without changing the selection algorithm. First, we check the sample count for each class in the union $(\mathcal{X}_c, \mathcal{Y}_c) \cup (\mathcal{X}_e, \mathcal{Y}_e)$. For classes with few or no samples, we add extra samples in descending order of their neighbor consistency c_i . This process continues until the count reaches $(1/M) \cdot N_c$, where N_c is the original size of that class in the noisy dataset and M is the total number of classes. Finally, we oversample minority classes to create a fully balanced set. In summary, there are various strategies to keep the training set balanced [44]; however, we will not elaborate on them further here. The resulting training set $(\mathcal{X}_t, \mathcal{Y}_t)$ is defined as:

$$(\mathcal{X}_t, \mathcal{Y}_t) = \text{Rebalancing}((\mathcal{X}_c, \mathcal{Y}_c) \cup (\mathcal{X}_e, \mathcal{Y}_e)). \quad (4)$$

3) Model Training: During the training phase, based on the training set $(\mathcal{X}_t, \mathcal{Y}_t)$ and $(\mathcal{X}, \mathcal{Y})$, we use the cross-entropy

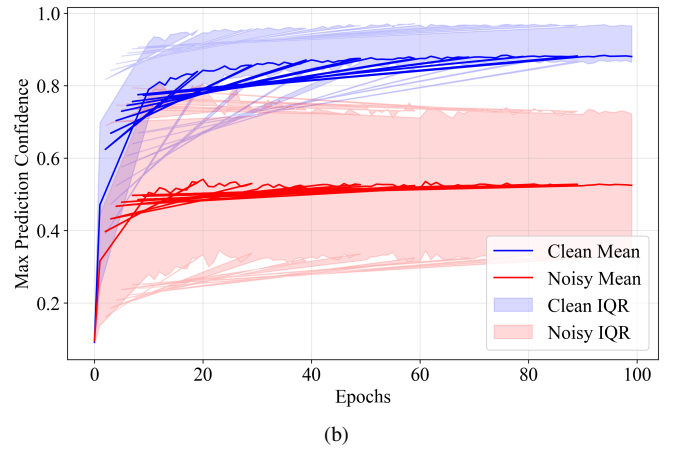
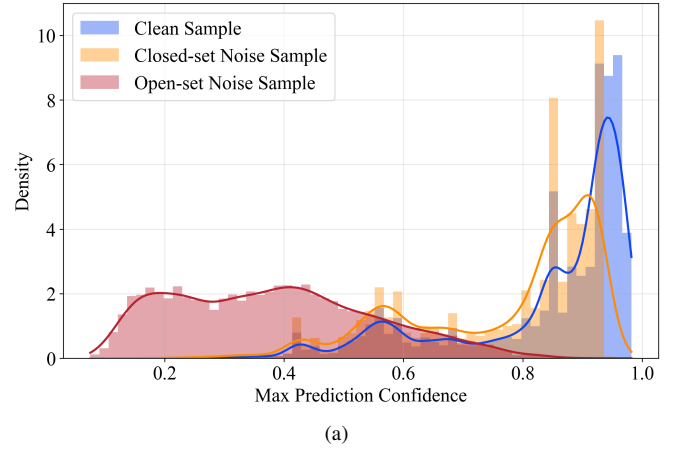


Fig. 5. In the Mal_TLS2023 dataset with 50% symmetric noise and 50% open-set noise: (a) Prediction confidence distribution for clean, closed-set noisy, and open-set noisy samples at epoch 20. The mean confidence values are 0.8421, 0.7746, and 0.3829, respectively. In this paper, we set the value of ζ to 0.93. (b) Evolution of maximum confidence over 100 epochs. The blue and red solid lines denote the per-epoch mean confidence for clean and noisy samples (including closed-set and open-set noise samples), respectively. The shaded areas represent the interquartile range (IQR, Q1–Q3) for each group, indicating the spread of the central 50% of samples.

loss function and self-supervised contrastive loss function to update the model $\mathcal{F}$ and the parameterized classifier $\mathcal{G}_p$.

Let $l \sim \text{Beta}(\alpha, \alpha)$, $l' = \max(l, 1 - l) \geq 0.5$. We concatenate two perspectives from the labeled dataset (obtained by swapping different feature columns) to get the sequence $\{(\mathcal{X}_k, \mathcal{Y}_k)\}_{k=1}^{2B}$, where B is the batch size and $2B$ is the total number of samples after concatenation. We apply a random permutation π to reorder the sequence, and then perform linear interpolation for the inputs and labels:

$$\tilde{x}_k = l'x_k + (1 - l')x_{\pi(k)}, \quad (5)$$

$$\tilde{y}_k = l'y_k + (1 - l')y_{\pi(k)}. \quad (6)$$

Here, $\tilde{x}_k$ and $\tilde{y}_k$ are the mixed samples and labels.

Based on the samples $\tilde{x}_k$ generated from MixUp [45], we calculate the supervised loss to optimize the classifier. First, we obtain the predicted probability distribution through the encoder $\mathcal{F}$ and classifier head $\mathcal{G}_p$:

$$p_k = \text{softmax}(\mathcal{G}_p(\mathcal{F}(\tilde{x}_k))) \in \mathbb{R}^M, \quad (7)$$

where M is the number of classes and p_k is a probability vector. Then, we calculate the cross-entropy loss, which measures the difference between the predicted probabilities p_k and the mixed labels $\tilde{y}_k$:

$$\mathcal{L}_{ce} = -\frac{1}{2B} \sum_{k=1}^{2B} \sum_{c=1}^M \tilde{y}_{k,c} \log p_{k,c}. \quad (8)$$

This loss is the average cross-entropy of all samples in the batch, where $\tilde{y}_{k,c}$ is the c -th component of the soft labels.

Considering that the original training set $(\mathcal{X}, \mathcal{Y})$ contains not only clean samples but also noisy samples, we ignore the labels and consider using self-supervised contrastive loss to optimize the feature space. Contrastive learning is employed to train the Sieve model to learn an embedding space where similar samples are close together, while dissimilar samples are far apart.

Let

$$v_i = \frac{\mathcal{H}(\mathcal{F}(u_i^{(1)}))}{\|\mathcal{H}(\mathcal{F}(u_i^{(1)}))\|_2}, \quad w_i = \frac{\mathcal{H}(\mathcal{F}(u_i^{(2)}))}{\|\mathcal{H}(\mathcal{F}(u_i^{(2)}))\|_2}, \quad i = 1, \dots, B, \quad (9)$$

where $u_i^{(1)}$ and $u_i^{(2)}$ represent the two views (or augmented views) of the i -th sample, processed by the neural network $\mathcal{F}$ to generate feature representations. $\mathcal{H}(z) \in \mathbb{R}^s, s = 128$, is a function to further process the features.

Let us define:

$$\begin{aligned} s_i^+ &= \frac{v_i^T w_i}{\tau}, \quad s_{ij}^{vw} = \frac{v_i^T w_j}{\tau}, \quad s_{ij}^{vv} = \frac{v_i^T v_j}{\tau}, \\ s_{ij}^{ww} &= \frac{w_i^T w_j}{\tau}, \quad s_{ij}^{ww} = \frac{w_i^T w_j}{\tau}, \end{aligned} \quad (10)$$

where these are measures of similarity between samples and τ is the temperature parameter. s_i^+ is the positive similarity between the same sample in different views, while s_{ij} denotes the negative similarities between different samples.

The objective of contrastive learning is to maximize the similarity between positive sample pairs (v_i, w_i) while minimizing the similarity between negative sample pairs $(v_i, w_j, i \neq j)$.

Let

$$D_i^{(1)} = e^{s_i^+} + \sum_{j \neq i} e^{s_{ij}^{vw}} + \sum_{j \neq i} e^{s_{ij}^{vv}}, \quad (11)$$

$$\text{loss}_{(v \rightarrow w)} = \frac{1}{B} \sum_{i=1}^B \left[-\log \frac{e^{s_i^+}}{D_i^{(1)}} \right]. \quad (12)$$

Here, $\text{loss}_{(v \rightarrow w)}$ represents the similarity between positive and negative samples, and $D_i^{(1)}$ is the normalization factor for the positive sample and all negative samples.

Similarly, we define

$$D_i^{(2)} = e^{s_i^+} + \sum_{j \neq i} e^{s_{ij}^{vw}} + \sum_{j \neq i} e^{s_{ij}^{ww}}, \quad (13)$$

$$\text{loss}_{(w \rightarrow v)} = \frac{1}{B} \sum_{i=1}^B \left[-\log \frac{e^{s_i^+}}{D_i^{(2)}} \right]. \quad (14)$$

The loss for the $w \rightarrow v$ direction is calculated similarly to $\text{loss}_{(v \rightarrow w)}$.

Thus, the overall contrastive loss is given by:

$$\mathcal{L}_{self} = \frac{1}{2} (\text{loss}_{(v \rightarrow w)} + \text{loss}_{(w \rightarrow v)}), \quad (15)$$

indicating that the contrastive loss is bidirectional, and the final symmetric loss is obtained by averaging $\text{loss}_{(v \rightarrow w)}$ and $\text{loss}_{(w \rightarrow v)}$.

Finally, the total loss $\mathcal{L}$ is a weighted sum of the supervised cross-entropy loss and the self-supervised contrastive learning loss:

$$\mathcal{L} = \mathcal{L}_{ce} + \lambda \mathcal{L}_{self}, \quad (16)$$

where λ is a hyperparameter that controls the balance between the supervised cross-entropy loss $\mathcal{L}_{ce}$ and the self-supervised contrastive loss $\mathcal{L}_{self}$.

C. Unknown Traffic Detection

After the training phase is completed, we hope that the Sieve model not only has the ability to classify known class traffic but also possesses the capability to detect unknown malicious traffic. Without changing the training mechanism of the Sieve model, we employ a non-parametric detection mechanism to accomplish the detection of unknown malicious traffic [46]. This is a plug-and-play module that can be applied to any other closed-world traffic classification model [42].

Specifically, when the Sieve model finishes training, under the optimal model parameters, the filtered clean training set is denoted as $(\mathcal{X}_t, \mathcal{Y}_t)$. We obtain the normalized feature set $\{\mathcal{F}(x_i)/\mathcal{F}(x_i)\|_2\}_{i=1}^{|\mathcal{X}_t|}$ and construct an index set based on this foundation. For the traffic sample to be detected $x' \in \mathbb{R}^d$, we first compute $z' = \mathcal{F}(x')$, and then calculate the minimum distance d from z' to the known class index set. If d is less than the threshold τ , the sample is classified as known class traffic; otherwise, it is classified as unknown class traffic. Formally, the anomaly score function can be expressed as:

$$\text{score}(z') = \min_M D(z', Z_M), \quad (17)$$

where $D(\cdot, \cdot)$ is a distance metric in the feature space, and Z_M represents the M clusters formed by the known classes. The threshold τ is set to the 95th percentile of the anomaly scores from the filtered clean training set. This strategy ensures that 95% of known class samples will not produce false positives, thus effectively controlling the misclassification of known traffic while maintaining high recall.

It should be noted that we use the Mahalanobis distance to compute the outlier score. Specifically,

$$\text{score}(z') = \min_M (z' - \mu_M)^T \Sigma_M^{-1} (z' - \mu_M), \quad (18)$$

where μ_M and Σ_M denote the sample mean and sample covariance of the features (z) from the known class clean training data. Through the eigendecomposition of the sample covariance, we have $\Sigma_M = Q_M \Lambda_M Q_M^{-1}$, and thus:

$$\text{score}(z') = \min_M (Q_M^T (z' - \mu_M))^T \Lambda_M^{-1} (Q_M^T (z' - \mu_M)), \quad (19)$$

which is equivalent to using the Euclidean distance scaled by the eigenvalues in the feature space.

In the case of Euclidean distance, that is, in the absence of scaling, components with higher eigenvalues receive greater weight but provide minimal discriminability. The scaling of eigenvalues eliminates this bias, making the Mahalanobis distance effective for detecting isolated points in the feature space.

D. Unknown Traffic Labeling

The detected unknown samples are defined as $\mathcal{D}_U = \{x_i^u\}_{i=1}^{|\mathcal{D}_U|}$. The labels of the samples in $\mathcal{D}_L = (\mathcal{X}_t, \mathcal{Y}_t)$ are as pure as possible but not completely correct. Our goal is to use automatic filtering and clustering of the samples in $\mathcal{D}_U$ to assist in the manual annotation of new samples. It is important to note that our unknown malicious traffic detection mechanism cannot ensure that all detected unknown traffic samples are from unknown categories. Hence, the labels y_i^u of the samples x_i^u in $\mathcal{D}_U$ either belong to known classes $\mathcal{C}^l = \{1, 2, \dots, M\}$ or come from new classes $\mathcal{C}^n \in N^+$. Therefore, our objective is to reassign misclassified samples back to $\mathcal{C}^l$ and cluster the remaining samples in $\mathcal{D}_U$.

1) *Unknown Traffic Clustering*: The samples in $\mathcal{D}_U$ can be divided into two parts:

$$\begin{aligned} \mathcal{D}_U &= \{x_i^u\}_{i=1}^{|\mathcal{D}_U|} = \mathcal{D}_l \cup \mathcal{D}_n \\ &= \{x_i^l \mid y_i^l \in \mathcal{C}^l\} \cup \{x_j^n \mid y_j^n \in \mathcal{C}^n\}, \end{aligned} \quad (20)$$

where $\mathcal{D}_l \cap \mathcal{D}_n = \emptyset$. All categories are denoted as $\mathcal{C} = \mathcal{C}^l \cup \mathcal{C}^n$ with $\mathcal{C}^l \cap \mathcal{C}^n = \emptyset$. However, we do not know the exact number $|\mathcal{C}^n|$. In this paper, we first estimate the value of $|\mathcal{C}^n|$ as $|\hat{\mathcal{C}}^n|$, and then use the samples and labels in $\mathcal{D}_L = (\mathcal{X}_t, \mathcal{Y}_t)$ as known information together with the semi-supervised K-means++ algorithm [43] to filter and cluster the samples in $\mathcal{D}_U$. The semi-supervised K-means++ algorithm first utilizes $\mathcal{D}_L$ to determine the cluster centers for $|\mathcal{C}^l|$ known classes, and selects the remaining $|\hat{\mathcal{C}}^n| = |\hat{\mathcal{C}}| - |\mathcal{C}^l|$ cluster centers based on the distance from features to the nearest cluster center. In each iteration, we enforce that the data in $\mathcal{D}_L$ maps to the true labels.

Finally, let the labels assigned to $\mathcal{D}_U = \{x_i^u\}_{i=1}^{|\mathcal{D}_U|}$ by the semi-supervised K-means++ algorithm be $\{\hat{y}_i^u\}_{i=1}^{|\mathcal{D}_U|}$, where $\hat{y}_i^u \in \mathcal{C}^l \cup N_{centroids}$, and $N_{centroids}$ denotes the new centroids for $|\hat{\mathcal{C}}^n|$ clusters. Samples in $\mathcal{D}_U$ belonging to clusters from $N_{centroids}$ and their corresponding predicted labels form $\mathcal{D}_U^l$.

Let $\mathcal{D}_U^l = \{(x_i^l, \hat{y}_i^l)\}_{i=1}^{|\mathcal{D}_U^l|}$, where $x_i^l \in \mathcal{D}_U$ and $\hat{y}_i^l \in N_{centroids}$. The goal of manual annotation is to correct and label each cluster.

At this point, network security engineers begin to intervene in the cluster labeling process. They will inspect the original traffic samples corresponding to each cluster (not just the abstract feature vectors) and identify and remove any obvious “anomalous” or “discordant” instances from each cluster. These discordant instances are manually moved to their rightful belonging clusters.

Since the unknown traffic detection model Sieve is not perfect, some instances that originally belong to known categories $\mathcal{C}^l$ may be misidentified as unknown traffic and mixed

into the new category set $\mathcal{D}_U^l$. Network security engineers need to identify these “fake new class” instances and remove them from $\mathcal{D}_U^l$ as much as possible to avoid label noise in subsequent learning.

If the estimated number of new categories $|\hat{\mathcal{C}}^n|$ does not match the reality:

- If the true number of categories is greater $|\mathcal{C}^n| > |\hat{\mathcal{C}}^n|$, new clusters need to be manually created to split the categories that have not been properly separated.
- If the true number is less $|\mathcal{C}^n| < |\hat{\mathcal{C}}^n|$, several clusters that should belong together must be merged.

Finally, a true Ground-Truth label is assigned to each corrected and semantically clear cluster, resulting in a final labeled dataset for incremental learning of new categories. This step significantly reduces the cost and workload of manual labeling. Labelers only need to perform efficient quality checks and fine-tuning, rather than starting from scratch with tedious organization work, hence greatly reducing labor costs.

2) *Class Number Estimation*: The purpose of estimating the number of categories is to determine the parameter k for the semi-supervised K-means++ algorithm, i.e., $|\mathcal{C}|$.

Inspired by [47]–[49], we do not know the exact number of $|\mathcal{C}|$, but we have a labeled sample set $\mathcal{D}_L = (\mathcal{X}_t, \mathcal{Y}_t)$. We can partition $\mathcal{D}_L$ into a probe set and a validate set.

Specifically, we first divide $\mathcal{D}_L = (\mathcal{X}_t, \mathcal{Y}_t)$ into a probe set $\mathcal{D}_L^{probe}$ with $\mathcal{C}_l^{probe}$ categories and a validate set $\mathcal{D}_L^{val}$ with $\mathcal{C}_l^{val}$ categories, where $\mathcal{C}_l^{probe} \cup \mathcal{C}_l^{val} = \mathcal{C}^l$, and $|\mathcal{C}_l^{probe}| : |\mathcal{C}_l^{val}| = 2 : 1$, and $\mathcal{C}_l^{probe} \cap \mathcal{C}_l^{val} = \emptyset$. Thus, $\mathcal{D}_L = \mathcal{D}_L^{probe} \cup \mathcal{D}_L^{val}$. We then launch the Brent algorithm [50] over the range $(|\mathcal{C}_l^{probe}|, |\mathcal{C}_l^{max}|]$ and use the semi-supervised K-means++ algorithm on $\mathcal{D}_L^{probe} \cup \mathcal{D}_L^{val} \cup \mathcal{D}_U$ until convergence. Here, $|\mathcal{C}_l^{max}|$ is the expected maximum total number of categories, which can be set to a larger value. During semi-supervised learning, samples in $\mathcal{D}_L^{probe}$ are forced to comply with the true labels, while samples in $\mathcal{D}_L^{val}$ are treated as additional “unlabeled” data. The clustering performance on $\mathcal{D}_L^{val}$ and $\mathcal{D}_U$ is assessed using average clustering accuracy (ACC) and the Davies-Bouldin index (DB index) [51] as follows. At the same time, Brent’s algorithm optimizes to maximize the ACC-DB objective. Finally, Brent’s algorithm terminates with the optimal estimation $|\hat{\mathcal{C}}|$. The class number estimation process of $\mathcal{D}_U$ is shown in Algorithm 1

Average Clustering Accuracy (ACC): ACC is applicable to labeled classes in the validate set $\mathcal{D}_L^{val}$ and is given by the formula:

$$ACC = \max_{\sigma \in S(\mathcal{C}_l^{val})} \left(\frac{1}{|\mathcal{D}_L^{val}|} \sum_{i=1}^{|\mathcal{D}_L^{val}|} \mathbf{1}\{y_i^{val} = \sigma(\hat{y}_i^{val})\} \right), \quad (21)$$

where y_i^{val} and $\hat{y}_i^{val}$ represent the true labels and the cluster labels assigned to feature x_i^{val} , respectively. $S(\mathcal{C}_l^{val})$ represents the permutation group of elements in $\mathcal{C}_l^{val}$. The ACC metric focuses on evaluating the clustering accuracy of labeled known data by solving the label matching problem to quantify clustering quality fairly.

Davies-Bouldin Index (DB): DB is an unsupervised clustering evaluation metric that does not require true labels, relying only on the data itself and the clustering results. The core idea of the DB index is to measure the compactness (similarity of samples within a cluster) and separability (differences between clusters). A smaller value indicates better clustering performance (tighter clusters, better separation between them), while a larger value indicates poorer clustering performance. It is defined as:

$$DB = \frac{1}{|\hat{\mathcal{C}}|} \sum_{i=1}^{|\hat{\mathcal{C}}|} \max_{j \neq i} \left(\frac{s_i + s_j}{d_{ij}} \right), \quad (22)$$

where the number of clusters is $\hat{\mathcal{C}}$, denoted as $\mathcal{C}_1, \mathcal{C}_2, \dots, \mathcal{C}_{|\hat{\mathcal{C}}|}$, and for each cluster $\mathcal{C}_i$, we define:

$$s_i = \frac{1}{|\mathcal{C}_i|} \sum_{x \in \mathcal{C}_i} \|x - \mu_i\|, \quad (23)$$

where μ_i is the centroid of cluster $\mathcal{C}_i$. The distance between clusters is measured as $d_{ij} = \|\mu_i - \mu_j\|$, where $\|\cdot\|$ typically denotes the Euclidean distance.

Algorithm 1 Class Number Estimation

Input: Labeled dataset $\mathcal{D}_L = (\mathcal{X}_t, \mathcal{Y}_t)$ with $|\mathcal{C}^l|$ known classes; unknown sample set $\mathcal{D}_U = \{x_i^u\}_{i=1}^{|\mathcal{D}_U|}$; maximum expected number of classes $|\mathcal{C}_l^{max}|$

Output: Estimated total number of classes $|\hat{\mathcal{C}}|$

- 1: Extract features of $\mathcal{D}_L$ and $\mathcal{D}_U$ using $\mathcal{F}$
 - 2: **Step 1: Split labeled dataset**
 - 3: Partition $\mathcal{D}_L$ by class into probe set $\mathcal{D}_L^{probe}$ and validate set $\mathcal{D}_L^{val}$ such that:
 - 4: 1) $\mathcal{D}_L = \mathcal{D}_L^{probe} \cup \mathcal{D}_L^{val}$
 - 5: 2) $|\mathcal{C}_l^{probe}| : |\mathcal{C}_l^{val}| = 2 : 1$
 - 6: 3) $\mathcal{C}_l^{probe} \cap \mathcal{C}_l^{val} = \emptyset$
 - 7: **Step 2: Define evaluation function $f(k)$**
 - 8: **function $f(k)$**
 - 9: 1. Apply semi-supervised K-means++ on dataset $\mathcal{D} = \mathcal{D}_L^{probe} \cup \mathcal{D}_L^{val} \cup \mathcal{D}_U$ with k clusters:
 - 10: - Constrain $\mathcal{D}_L^{probe}$ samples to their true labels
 - 11: - Treat $\mathcal{D}_L^{val}$ and $\mathcal{D}_U$ as unlabeled
 - 12: 2. After convergence, compute:
 - 13: - ACC: Average clustering accuracy on $\mathcal{D}_L^{val}$
 - 14: - DB: Davies-Bouldin index on $\mathcal{D}$
 - 15: 3. **return** $f(k) = -(\text{ACC} - \text{DB})$
 - 16: **end function**
 - 17: **Step 3: Find optimal k using Brent's algorithm**
 - 18: Run Brent's algorithm on interval $(|\mathcal{C}_l^{probe}|, |\mathcal{C}_l^{max}|]$ to minimize $f(k)$
 - 19: Let k^* be the k that minimizes $f(k)$
 - 20: Set optimal estimate $|\hat{\mathcal{C}}| = k^*$
 - 21: **return** $|\hat{\mathcal{C}}|$
-

V. THEORETICAL JUSTIFICATION

In this section, we provide a theoretical analysis of the proposed Sieve framework from an information-theoretic perspective. Specifically, we prove three theorems to elucidate how

Sieve prevents information collapse, avoids label blindness, and optimizes intra-class compactness for effective unknown traffic detection¹.

Theorem 1. (Proof in Appendix A): Let Z_{ce} be the feature representation learned by minimizing only the cross-entropy loss, and Z_{sieve} be the representation learned by Sieve with a self-supervised contrastive loss weight $\lambda > 0$. Under the idealized Information Bottleneck framework, as the compression coefficient $\beta \rightarrow \infty$, Z_{ce} converges to a minimal sufficient statistic of Y , resulting in the asymptotic loss of detection-relevant information, i.e., $I(Z_{ce}; T|Y) = 0$. Conversely, under the assumption of non-vanishing structural information, the optimization objective of Sieve provides a theoretical basis for a strictly positive lower bound for detection information, i.e., $I(Z_{sieve}; T|Y) > 0$.

Theorem 2. (Proof in Appendix B): Let $\mathcal{Y}_s$ be the proxy target for self-supervised learning. Assuming the Label Blindness Condition $I(\mathcal{Y}_s; Y) = 0$ holds, the unsupervised feature Z_{ssl} satisfies $I(Z_{ssl}; Y) = 0$. Sieve aims to overcome this blindness condition by introducing the neighbor-consistency corrected label Y_c . Under the Manifold Smoothness Assumption and the condition of high-purity filtering, $I(Z_{sieve}; Y)$ asymptotically approaches the label entropy $H(Y)$, thus enabling the detection of adjacent unknown samples.

Theorem 3. (Proof in Appendix C): Let Z follow a class-conditional distribution with covariance matrix Σ_k . The self-supervised optimization objective of Sieve ($\mathcal{L}_{self}$) is mathematically equivalent to minimizing the trace of the intra-class covariance matrix Σ_k , i.e., $\text{Tr}(\Sigma_k) \rightarrow 0$. Under the assumption of a class-conditional Gaussian distribution, this implies that the determinant $\det(\Sigma_k) \rightarrow 0$. In the asymptotic limit, this compactness causes the eigenvalue amplification factor of the Mahalanobis distance metric to tend towards infinity, thereby minimizing the theoretical bound of the Open Set Risk.

The empirical findings presented in Section VI provide robust validation for the information-theoretic foundations of the Sieve framework. Consistent with Theorem 1, ROC analysis demonstrates that Sieve achieves AUC values reaching 99.92%, significantly outperforming state-of-the-art methods in unknown threat detection scenarios. Furthermore, the experimental results substantiate Theorem 2 as the framework successfully overcomes label blindness; this is evidenced by the direct positive correlation between label accuracy and model generalization, combined with a robust capability to reject open-set noise. Finally, the intra-class compactness predicted by Theorem 3 is visually confirmed by t-SNE visualizations where known categories form well-defined and compact clusters; this geometric refinement results in the distinct gap observed in anomaly score distributions, allowing Mahalanobis-based detection to minimize open-set risk more effectively than variants like Sieve-MSP or Sieve-Energy.

¹See repository <https://github.com/niebiakong/Sieve/tree/master/theory>

VI. EVALUATION

In this section, we conducted comprehensive experiments on multiple publicly available traffic datasets to demonstrate the superiority and effectiveness of our proposed Sieve model. Specifically, these experiments aim to address the following questions:

- **RQ1.** How does the Sieve model perform in detecting known malicious traffic compared to SOTA methods under different noise conditions?
- **RQ2.** How does the Sieve model perform in detecting unknown malicious traffic compared to SOTA methods under different noise conditions?
- **RQ3.** What is the effectiveness of the analysis of detected unknown traffic samples?

A. Setup

1) *Dataset:* The experiments are based on four public datasets.

Mal_TLS2023 [52]: This dataset contains TLS 1.2 encrypted traffic generated by 23 malware families. We segment raw packet traces into flows using the standard five-tuple and extract one-dimensional statistical features from each TLS flow. The feature extraction code is available in our repository². After processing, the dataset contains 92,034 flows.

CipherSpectrum [53]: This dataset contains TLS 1.3 encrypted web traffic using three mandatory/recommended cipher suites: AES-128-GCM, AES-256-GCM, and CHACHA20-POLY1305. Each traffic type contains 2,010 flows, resulting in 82,410 flows in total. Since it is also TLS encrypted traffic, we apply the same feature extraction pipeline as Mal_TLS2023.

DDoS2019 [54]: This dataset includes various reflective DDoS attacks, such as PortMap, NetBIOS, LDAP, MSSQL, and SNMP. We treat all attack traffic as malicious and real background traffic as benign. We use CICFlowMeter³ to extract flow-level statistical features. After processing, it contains 523,831 attack flows and 106,626 benign flows, totaling 630,457 flows.

IDS2018 [55]: We use traffic from two days, i.e., 3.2 and 2.16, and select DoS attacks-Hulk, DoS attacks-SlowHTTPTest, and Bot as malicious traffic, while treating the remaining benign traffic as normal. Following DDoS2019, we use CICFlowMeter to extract the same statistical features. The processed dataset contains 1,074,342 benign flows and 290,089 malicious flows, totaling 1,364,431 flows.

2) *Noise Settings:* To simulate realistic data corruption, we include three noise types: symmetric, asymmetric, and open-set. Symmetric noise is created by randomly replacing labels across all classes using a uniform distribution. In contrast, asymmetric noise models semantic confusion by swapping labels between similar categories. For example, in the CipherSpectrum dataset, traffic from `googleapis.com` or `googletagmanager.com` is randomly relabeled as `google.com`. Open-set noise occurs when samples from

external datasets are incorrectly assigned labels from the current set. We define the total noise ratio as the overall proportion of noisy samples in the dataset, while the open-set noise ratio refers to the portion of open-set samples within the noisy subset. For brevity, experimental settings are denoted by their ratios and types: for instance, “0.3Sym” and “0.4Asym” represent 30% symmetric and 40% asymmetric noise, respectively, while “0.3Sym_0.5Open” indicates a 30% total noise ratio with a 50% open-set component.

3) *Experimental Settings:* The experimental setup is summarized in Table III. For the Mal_TLS2023 dataset, we use three randomly selected types of traffic from CipherSpectrum as open-set noise samples, and all categories in CipherSpectrum as unknown traffic. For the DDoS2019 dataset, we use benign traffic from IDS2018 as open-set noise samples and all traffic from IDS2018 as unknown traffic.

TABLE III
EXPERIMENTAL ENVIRONMENT SETUP.

Characteristics	Mal_TLS2023	DDoS2019
# Attack types	23	DDoS variants
# Normal samples	0	106,626
# Attack samples	92,034	523,831
Task	Multi-class	Binary
Open-set noise classes	Randomly select 3 classes	Benign
Unknown classes	CipherSpectrum	IDS2018

4) *Compared Methods:* To comprehensively evaluate Sieve, we compare it with state-of-the-art methods from two closely related but distinct research lines, since there is no published baseline specifically designed for the joint setting of mixed label noise and unknown traffic detection.

- **Noise-Robust Learning Methods** (e.g., DSDIR [14], MCRé [15], and RAPIER [16]) are designed to learn from training sets corrupted by closed-set label noise, and are mainly evaluated on known-class classification performance. Comparing Sieve with them verifies its robustness to noisy labels in known traffic classification.
- **Unknown Attack Detection Methods** (e.g., AEGIS-Net [5], FOSS [18], and RFG-HELAD [22]) aim to identify samples from unseen classes, but usually assume a clean training set. Comparing Sieve with them evaluates its ability to detect unknown malicious traffic in the presence of label noise.

For a fair comparison, all baseline methods are evaluated under the same experimental protocol as Sieve, using the same flow-level feature vectors, preprocessing pipeline, train/test split, and noise injection settings. Although some baselines were originally designed with different feature extraction procedures, their inputs can be represented as one-dimensional feature vectors; thus, we use the unified feature representation for all compared methods to avoid biases caused by inconsistent inputs or data partitions. We follow the original papers or publicly available implementations of the baselines as closely as possible and retain their default parameter settings. In Sieve, the CNN-based deep residual network is used as the feature encoder to learn compact and robust representations.

²<https://github.com/niebiakong/Sieve>

³<https://github.com/ahlashkari/CICFlowMeter>

TABLE IV
THE CLOSED-WORLD PERFORMANCE OF ALL METHODS UNDER DIFFERENT NOISE RATIOS ON THE MAL_TLS2023 DATASET.

Methods	0.1Sym_0.5Open		0.3Sym_0.5Open		0.5Sym_0.5Open		0.1Asym_0.5Open		0.3Asym_0.5Open		0.5Asym_0.5Open	
	Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1
RAPIER	86.54	85.66	83.26	83.78	81.43	81.32	85.73	85.56	84.10	84.57	80.74	80.39
MCRc	85.23	83.66	69.72	67.24	54.53	49.73	88.00	86.98	81.09	79.12	74.25	72.28
DSDIR	94.52	93.75	94.17	92.86	93.53	92.23	94.82	93.65	94.08	92.51	93.73	92.40
FOSS	70.45	63.14	74.11	69.39	71.19	64.85	84.26	82.09	76.17	71.05	69.72	63.57
RFG-HELAD	91.04	89.19	91.37	89.60	91.23	89.65	91.79	90.61	91.88	90.54	92.60	91.82
CNN	93.94	93.82	91.38	91.13	90.91	90.67	93.29	93.16	91.99	91.89	89.36	89.34
Sieve	95.45	95.19	95.40	95.12	94.96	94.75	94.85	94.59	95.20	94.95	95.25	95.01

Note: The CNN model represents the model without the noise correction module in the Sieve, trained only with the cross-entropy loss function. The noise setting format is [total noise ratio][noise type]_[open-set noise ratio]Open.

TABLE V
THE CLOSED-WORLD PERFORMANCE OF ALL METHODS UNDER DIFFERENT NOISE RATIOS ON THE DDoS2019 DATASET.

Methods	0.1Sym_0.5Open		0.3Sym_0.5Open		0.5Sym_0.5Open		0.1Asym_0.5Open		0.3Asym_0.5Open		0.5Asym_0.5Open	
	Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1
RAPIER	76.66	84.73	78.87	86.48	72.42	82.02	70.86	80.24	67.06	77.71	68.06	78.03
MCRc	96.59	96.49	89.91	89.09	74.87	64.12	94.95	94.69	85.16	83.31	74.76	70.60
DSDIR	87.33	65.81	84.97	54.75	83.35	45.52	91.60	87.27	88.75	83.75	89.45	84.59
FOSS	82.93	75.19	82.93	83.90	77.42	75.19	84.54	78.79	83.86	77.33	84.72	79.19
RFG-HELAD	97.93	97.93	97.93	97.93	97.72	97.72	97.93	97.93	97.93	97.93	97.89	97.89
CNN	94.74	94.78	89.94	89.84	86.77	86.12	95.75	95.86	88.87	88.90	86.82	86.23
Sieve	98.48	98.48	98.51	98.51	98.58	98.58	99.12	99.12	99.07	99.07	99.05	99.06

The noise setting format is [total noise ratio][noise type]_[open-set noise ratio]Open.

5) *Training Details*: The Sieve model utilizes a CNN-based deep residual neural network for feature extraction, with a standard SGD optimizer featuring a momentum of 0.9 and a weight decay of $5e-4$. The learning rate is set to 0.001 with a cosine learning rate schedule, and the total training epoch is 100. The batch size is 256. The parameters ξ and ζ are selected from $\{0.91, 0.92, \dots, 1.0\}$. The parameters K in KNN voting and the λ are chosen via grid search on a held-out validation set, with $K \in \{10, 20, \dots, 100, 150, 200\}$ and $\lambda \in \{0.1, 0.2, \dots, 1.0\}$. The optimal values, determined by maximizing validation performance, are $\xi = 1.0$, $\zeta = 0.93$, $K = 100$, and $\lambda = 1.0$. All reported results are averaged over experiments conducted with five different random seeds to ensure robustness.

B. Known Malware Traffic Classification Experiments (RQ1)

To comprehensively evaluate the classification performance of the Sieve model on known malware traffic in a closed-world scenario, especially in a complex environment characterized by the simultaneous presence of closed-set noise and open-set noise, we compared Sieve with current SOTA methods. The comparison methods include: noise label handling methods (RAPIER, MCRc, DSDIR); unknown traffic detection methods (FOSS, RFG-HELAD); and a baseline model (standard CNN neural network without a noise processing module). Through the analysis of Table IV and Table V, we arrived at the following in-depth conclusions:

(1) **Multiclass Classification Performance Analysis**. Experimental results on the Mal_TLS2023 multi-classification dataset demonstrate that Sieve possesses significant advantages

in handling mixed noise. As shown in Table IV, the Sieve model achieved optimal performance under all noise settings (both symmetric and asymmetric), with average Accuracy and F1 scores exceeding 94%. Notably, even under extremely high noise rates—such as the 0.5Sym_0.5Open scenario, where 50% of labels are incorrectly flipped, and 50% of unknown class samples are mixed in—Sieve’s performance experienced only a minor decline. This proves that its neighbor consistency selection mechanism and subset expansion strategy can effectively isolate high-confidence clean samples within the feature space. Regarding the standard CNN, it shows a certain degree of resistance to label noise. Among the noise label handling methods, DSDIR performs best because it is a two-stage method where each stage promotes a purer training subset. Conversely, the MCRc method gradually loses effectiveness in high-noise scenarios. FOSS and RFG-HELAD perform relatively poorly as they lack noise label processing modules; however, RFG-HELAD outperforms FOSS because it incorporates frequency domain features and adversarial training mechanisms, which enhance the robustness of feature expression.

(2) **Binary Classification Performance Analysis**. In the DDoS2019 binary classification task, the data distribution imbalance and the characteristics of attack traffic make the task unique. As shown in Table V, Sieve’s accuracy remains stable above 98.4% across all scenarios, and even reached 99.06% under the 0.5Asym extreme asymmetric noise setting. This indicates that Sieve’s sample screening mechanism is not only suitable for multi-classification but is also extremely precise in binary boundary determination, remaining unaffected by open-

set background traffic interference. A significant finding is that while DSDIR performs excellently in multi-classification, its performance collapses severely in binary classification under high noise scenarios. RFG-HELAD demonstrates strong stability in this task, second only to Sieve. This is attributed to its heterogeneous ensemble learning architecture, which effectively captures the inherent patterns of attack flows when processing large-scale DDoS traffic, maintaining good discriminability even in the presence of label noise.

(3) **Label Correction Capability Analysis.** The core advantage of Sieve lies in its powerful noise label correction and screening capabilities, which directly determine the performance of the subsequent classifier. Fig. 6 illustrates the dynamic screening of the training subset by Sieve during the training process on the Mal_TLS2023 dataset. In the initial stage of training, the model mainly relies on initial features; as the Epoch increases, the proportion of the purple area (closed-set noise samples) decreases significantly and continuously. For example, in the 0.3Sym_0.5Open scenario, the proportion of closed-set noise drops substantially, while the corresponding blue area (clean samples) increases significantly. This indicates that Sieve successfully identified and corrected the mislabeled known-class samples. It is worth noting that the proportion of the red area (open-set noise samples) remains almost constant throughout the entire training cycle and is never incorrectly “corrected” into clean samples. This proves that Sieve possesses an extremely strong rejection capability—it not only corrects wrong labels but also identifies open-set noise that does not belong to any known category and excludes it from model updates, thereby preventing open-set data from polluting the decision boundary.

Fig. 7 displays the synchronous trend of Label Accuracy and Testing Accuracy. As training progresses, the label correction module continuously improves the purity of the training data, directly driving the performance improvement of the model on the test set. This positive correlation strongly proves that Sieve’s label correction module is a key component for enhancing the model’s generalization ability.

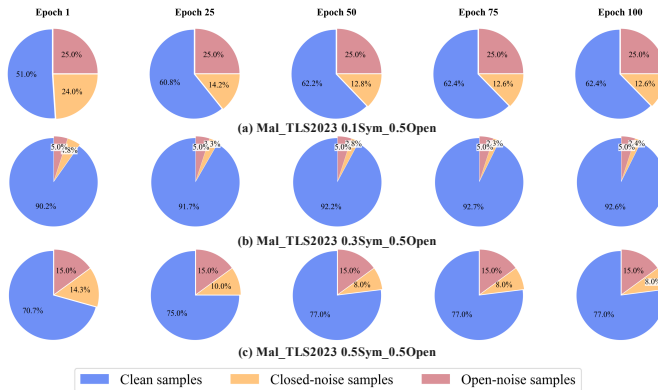


Fig. 6. The changes in the proportions of three types of samples during the training process of the Sieve model on the Mal_TLS2023 dataset.

(4) **Confusion Matrix Analysis.** The confusion matrix for the 0.1Sym_0.5Open scenario (Fig. 8) exhibits a distinct diagonal-dominant pattern, reflecting high classification

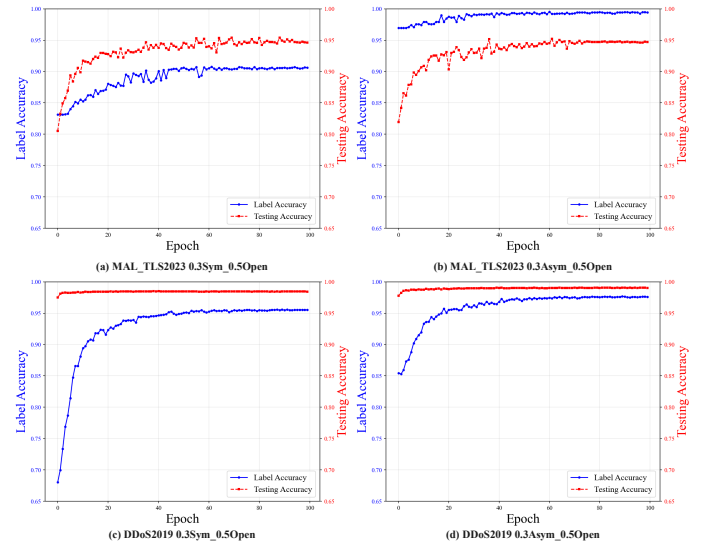


Fig. 7. The changes in label accuracy and model test accuracy across different scenarios with respect to the number of training epochs.

accuracy across most categories. Despite the presence of 50% open-set noise and 10% label flipping, Sieve effectively extracts core features of malicious traffic without significant category confusion. Notably, minor mutual misclassification between Class 6 and Class 7 arises from their inherent semantic similarity in TLS-encrypted traffic features rather than noise-induced chaos. This localized confusion confirms that Sieve’s feature learning preserves the clustering structure and maintains robust discriminability, with misclassifications limited to edge samples with highly similar characteristics.

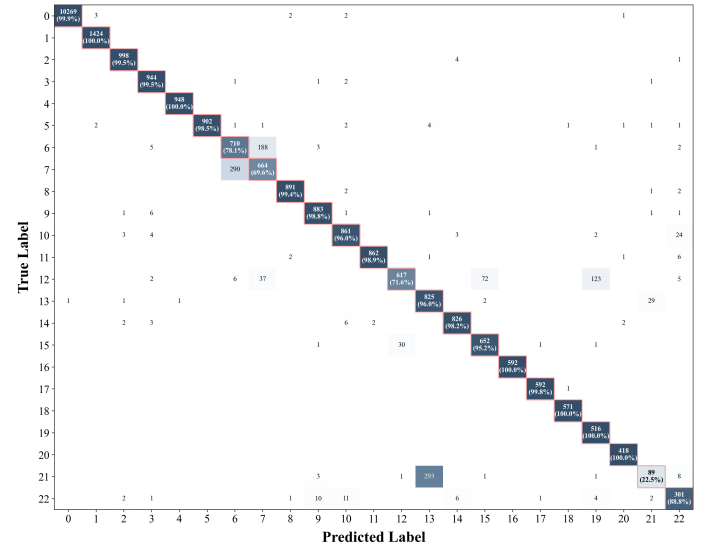


Fig. 8. The confusion matrix of the Sieve model in the 0.1Sym_0.5Open scenario on the Mal_TLS2023 dataset.

C. Unknown Malware Traffic Detection Experiments (RQ2)

To evaluate the capability of the Sieve model in detecting unknown malicious traffic, we conducted a series of experiments. The following performance analysis regarding Sieve’s

TABLE VI
THE OPEN-WORLD PERFORMANCE OF ALL METHODS UNDER DIFFERENT NOISE RATIOS ON MAL_TLS2023 AND DDoS2019 DATASET.

Dataset	Methods	0.1Sym_0.5Open		0.3Sym_0.5Open		0.5Sym_0.5Open		0.1Asym_0.5Open		0.3Asym_0.5Open		0.5Asym_0.5Open	
		Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1
Mal_TLS2023	FOSS	97.02	95.38	52.40	56.36	40.21	50.71	98.36	97.40	74.33	70.55	49.73	54.98
	RFG-HELAD	85.93	87.69	88.13	89.41	88.08	89.37	80.44	83.67	84.79	86.82	87.29	88.75
	AEGIS-Net	96.64	94.52	97.07	96.73	85.25	80.79	95.60	92.70	97.80	96.48	98.19	97.13
	Sieve-MSP	77.50	74.06	90.20	86.78	72.02	69.68	61.99	62.85	73.22	70.58	83.47	79.57
	Sieve-Energy	82.83	78.19	87.46	83.13	81.93	77.77	73.58	68.52	82.07	77.99	86.62	81.86
	Sieve-Cosine	84.38	76.40	82.73	76.35	81.03	73.14	82.78	76.83	84.15	83.52	76.89	79.57
	Sieve	97.57	96.10	98.05	96.89	98.19	97.12	97.20	95.48	97.87	96.60	98.12	97.00
DDoS2019	FOSS	97.68	96.09	92.75	88.72	53.41	55.02	85.87	80.13	67.56	63.75	33.47	46.14
	RFG-HELAD	61.66	71.25	60.98	70.89	61.37	71.09	60.30	70.53	60.23	70.49	61.00	70.90
	AEGIS-Net	52.28	53.98	36.44	47.00	33.50	45.94	49.97	53.32	37.13	47.72	33.16	46.01
	Sieve-MSP	95.11	92.09	94.16	90.85	94.49	91.31	91.37	86.74	94.53	91.28	95.06	92.23
	Sieve-Energy	90.55	82.95	89.97	83.42	85.69	77.12	90.85	84.42	89.15	80.83	86.25	73.71
	Sieve-Cosine	74.65	60.49	77.34	58.68	74.29	57.51	72.55	61.10	72.08	59.01	73.10	55.18
	Sieve	98.81	97.98	99.23	99.62	99.54	99.92	99.15	98.57	98.61	98.44	98.12	98.12

Note: Acc and F1 are calculated after determining the detection threshold τ . Sieve-MSP, Sieve-Energy, and Sieve-Cosine are variants of Sieve that replace the Mahalanobis distance metric with different OOD scoring mechanisms.

Sieve-MSP: Uses the Maximum Softmax Probability of the classifier's logits to measure detection confidence [56].

Sieve-Energy: Calculates the Energy score based on the LogSumExp of the output logits [57].

Sieve-Cosine: Measures the maximum cosine similarity between the normalized test sample feature and the training feature bank [58].

unknown traffic detection capabilities is derived from the results:

(1) **Comparative Analysis of Unknown Traffic Detection Performance.** Table VI presents the performance of various models in open-world scenarios under different noise settings on the Mal_TLS2023 and DDoS2019 datasets. Across all tested noise scenarios, Sieve consistently achieved the highest Accuracy and F1 scores. Notably, under high noise ratios, Sieve maintained F1 scores above 97% (Mal_TLS2023) and 98% (DDoS2019), demonstrating exceptional robustness. Although FOSS is specifically designed for unknown traffic detection, it performed poorly on training sets containing label noise. Its performance degraded significantly as the noise ratio increased; for instance, on Mal_TLS2023, the F1 score dropped from 97% to approximately 50%. This indicates that FOSS lacks an effective noise filtering mechanism, causing the boundaries of “known classes” to be severely distorted by noise. AEGIS-Net and RFG-HELAD performed acceptably under low noise conditions but struggled to effectively distinguish between known and unknown classes under high or asymmetric noise, resulting in a decline in detection rates. Furthermore, Sieve outperformed its variants—Sieve-MSP, Sieve-Energy, and Sieve-Cosine. This substantiates that utilizing the Mahalanobis distance for anomaly detection, following the compaction of the feature space, is the most effective strategy.

(2) **Analysis of ROC Curves and AUC Metrics.** Fig. 9 illustrates the ROC curves for unknown traffic detection by various models under different datasets and noise settings. The ROC curve of Sieve (solid red line) is consistently positioned in the upper-left corner, encompassing the largest area (AUC value). For example, under the 0.1Sym_0.5Open setting of Mal_TLS2023, Sieve achieved an AUC of 99.92%, surpassing comparative methods such as FOSS and RFG-HELAD. The

morphology of the ROC curves indicates that Sieve can attain a very high True Positive Rate (TPR) while maintaining an extremely low False Positive Rate (FPR). This implies that Sieve can accurately identify the vast majority of unknown attacks without frequently misclassifying normal traffic or known attacks. In contrast, the detection capabilities of RFG-HELAD and AEGIS-Net in novel attack intrusion detection scenarios approach random guessing.

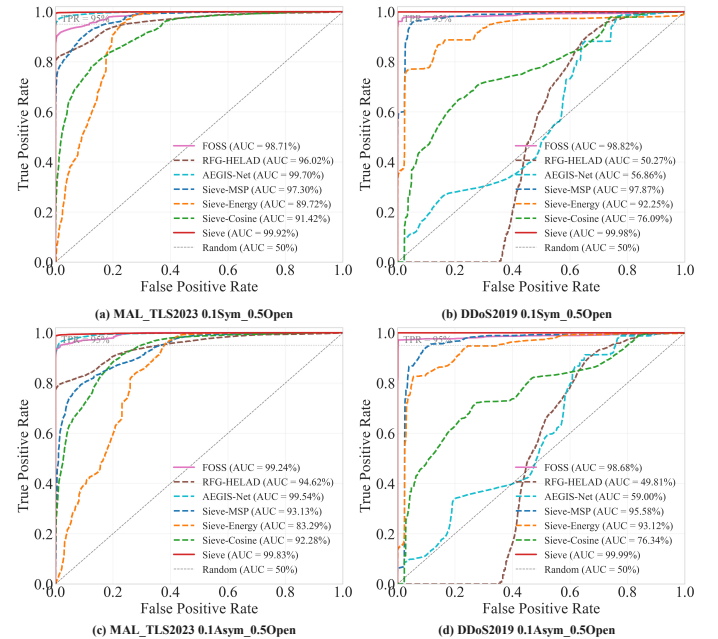


Fig. 9. ROC curves for the unknown malware traffic detection in Mal_TLS2023 and DDoS2019 dataset.

(3) **Analysis of Anomaly Score Distribution and Thresholds.** Fig. 10 visually presents the distribution of anomaly

scores calculated by Sieve for known and unknown traffic. It is clearly observable that the anomaly scores for known traffic are concentrated in the lower numerical range (left blue region) with a highly compact distribution, whereas the anomaly scores for unknown traffic are distributed in the higher numerical range (right orange region). There is a distinct “gap” or minimal overlap between the two. The dashed line in the figure represents the detection threshold, which successfully isolates the vast majority of unknown traffic to the right. For instance, in the 0.1Sym_0.5Open scenario of Mal_TLS2023, the threshold effectively distinguishes most unknown traffic. This demonstrates that Sieve, by optimizing intra-class compactness, causes known classes to be extremely convergent in the feature space, thereby enabling the easy detection of unknown classes as anomalies via a distance threshold.

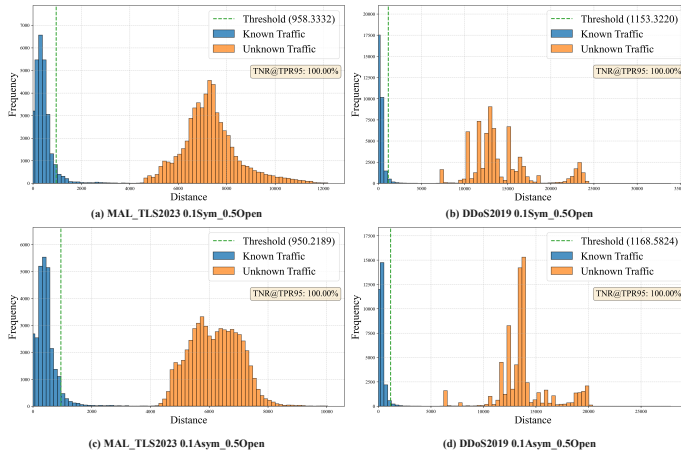


Fig. 10. Distribution of anomaly scores.

(4) **False Positive Analysis.** Table VII shows the impact of the TPR parameter in the Sieve framework on controlling the false positive rate (FPR). Increasing the TPR can systematically and significantly reduce the FPR. For example, in the 0.1Sym_0.5Open noise scenario of the MalTLS2023 dataset, raising the TPR from 0.95 to 1.0 reduces the FPR dramatically from 3.56% to 0.58%. This pattern holds consistently across all tested noise configurations. This occurs because the TPR directly determines the strictness of the model’s decision threshold for distinguishing known classes from unknown classes. Increasing the TPR implies adopting a stricter threshold, thereby more effectively protecting known-class samples from being misclassified as unknown threats, which in turn directly reduces false positives. This design provides crucial operational value for practical deployment: in production environments where alert-handling resources are limited and tolerance for false positives is low, operators can proactively set a high TPR to suppress the FPR to an extremely low level, ensuring the manageability of alert volume [59], [60].

(5) **t-SNE Visualization Analysis of Feature Space.** Fig. 11 visualizes the distribution of known and unknown classes in the feature space by mapping high-dimensional features onto a two-dimensional plane using the t-SNE algorithm [61], [62]. The visualization results show that after Sieve training,

TABLE VII
FPR UNDER DIFFERENT NOISE SCENARIOS WITH TUNABLE TPR THRESHOLD.

Datasets	Scenarios	TPR			
		0.95	0.97	0.99	1.0
Mal_TLS2023	0.1Sym_0.5Open	3.56	2.80	2.52	0.58
	0.3Sym_0.5Open	3.35	2.32	1.03	0.18
	0.5Sym_0.5Open	3.78	3.11	3.00	2.09
	0.1Asym_0.5Open	3.57	2.72	2.34	1.25
	0.3Asym_0.5Open	3.37	2.83	2.48	1.16
	0.5Asym_0.5Open	3.15	2.90	2.96	0.35
DDoS2019	0.1Sym_0.5Open	9.35	6.76	3.89	0.56
	0.3Sym_0.5Open	8.64	6.07	3.35	0.39
	0.5Sym_0.5Open	6.95	4.77	2.35	0.08
	0.1Asym_0.5Open	6.15	4.12	1.62	0.06
	0.3Asym_0.5Open	6.12	3.89	1.40	0.01
	0.5Asym_0.5Open	6.49	4.09	1.53	0.00

each known category (clusters of different colors) clusters very tightly in the feature space, forming distinct, well-defined clusters. This is attributed to the compression of intra-class distances by the contrastive learning loss function. Unknown traffic is effectively pushed away from the distribution regions of known classes and scattered in low-density areas of the feature space. Compared to models without Sieve optimization, Sieve maintains a clear structure of the feature space even under high noise interference, avoiding the blurring of category boundaries caused by noisy samples. This provides a solid feature foundation for distance-based unknown traffic detection.

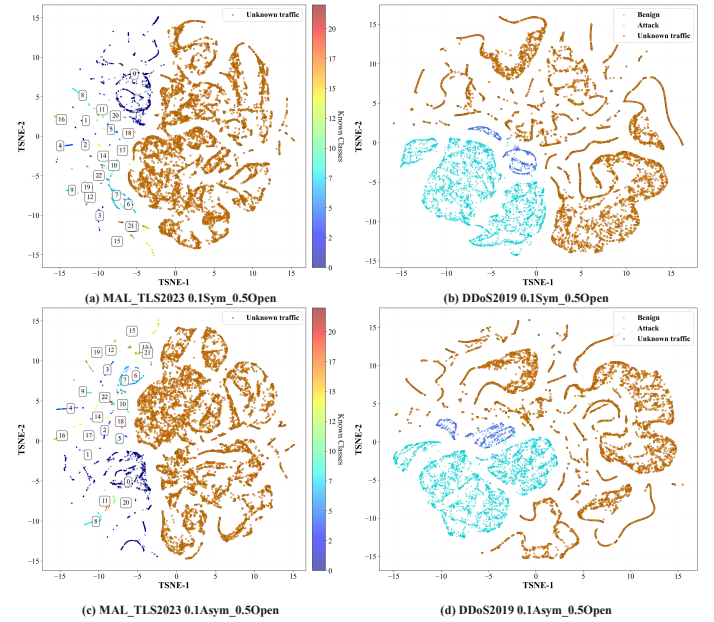


Fig. 11. Comparison of t-SNE.

D. Class Number Estimation and Clustering Analysis (RQ3)

To validate the efficacy of the Sieve framework during the unknown traffic labeling phase, we conducted class number estimation and clustering experiments on the IDS2018 dataset.

The unknown traffic detector is trained in the DDoS2019 dataset under the 0.1Sym_0.5Open scenario.

TABLE VIII
DESCRIPTION OF THE UNKNOWN TRAFFIC SELECTED FROM THE IDS2018 DATASET.

Attack Name	Date	Attack Start Time	Attack Finish Time
DoS-Hulk	Fri-16-02-2018	13:45	14:19
DoS-SlowHTTPTest	Fri-16-02-2018	10:12	11:08
Bot	Fri-02-03-2018	10:11	11:34
Bot	Fri-02-03-2018	14:24	15:55

(1) **Analysis of Class Number Estimation Accuracy.** In this experiment, the DDoS2019 dataset was employed as the known traffic, while traffic from two specific days—February 16, 2018 (Friday) and March 2, 2018 (Friday)—within the IDS2018 dataset was selected as the unknown traffic. As indicated in the Table VIII, the attack traffic on these two days primarily consists of DoS-Hulk, DoS-SlowHTTPTest, and Bot. According to the official dataset documentation, the Bot traffic comprises two distinct botnet variants: Zeus and Ares. Consequently, including the background benign traffic, the total number of Ground Truth classes in this experimental scenario is 5 (4 attack classes + 1 benign class).

In the experiment, we set the upper bound of the search to $|C_l^{max}| = 10$. The Prior-Guided Class Number Estimation module of Sieve utilizes the Brent algorithm to optimize the ACC-DB objective function, ultimately estimating the number of unknown traffic classes to be 6. Although there is a deviation of one class compared to the ground truth (5 classes), this indicates that Sieve tends towards a finer-grained division (over-segmentation). In the field of cybersecurity, the cost of misclassifying variants of the same attack into two classes is significantly lower than that of confusing two distinct attacks into a single class (under-segmentation). Therefore, this estimation result is acceptable in practical applications and facilitates subsequent manual merging.

(2) **In-depth Analysis of Clustering Performance Metrics.** Based on the estimated class number $k = 6$, we employed the semi-supervised K-means++ algorithm to cluster the unknown traffic encoded by the encoder. Fig. 12 presents the detailed quantitative evaluation metrics. The clustering accuracy of the Sieve model for known categories approaches 100%. This demonstrates that while exploring unknown categories, Sieve avoids catastrophic forgetting and perfectly retains its discriminative capability for known traffic. The Novel Acc is 0.418; the primary reason for this relatively low metric is that the estimated number of classes (6) exceeds the ground truth (5). This results in a specific class of real attack traffic being split into two clusters, thereby incurring a penalty during the purity calculation. The harmonic mean of the accuracy for known and unknown classes is 0.589. This metric comprehensively reflects the clustering performance of the model across the entire feature space, indicating that the model achieves a balance between maintaining performance on known classes and discovering new ones.

NMI (Normalized Mutual Information) is used to measure the correlation between clustering results and ground truth labels. The Overall NMI (0.704) is at a relatively high level, indicating that the clustering results are highly consistent with the true distribution. The Novel NMI (0.215) is relatively lower, which is similarly constrained by the cluster splitting resulting from the overestimation of the class number. The ARI (Adjusted Rand Index) measures the difference between the clustering results and a random partition. A score of 0.656 indicates that Sieve's clustering results are significantly superior to random guessing and effectively aggregate similar flows.

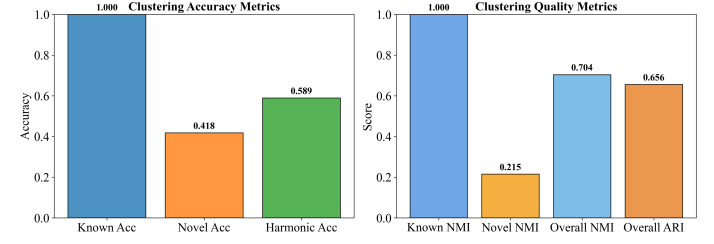


Fig. 12. Performance evaluation of unknown traffic clustering on the IDS2018 Dataset in terms of accuracy and quality Metrics.

(3) **t-SNE Visualization Analysis.** To intuitively demonstrate the feature learning and clustering effectiveness of Sieve for unknown traffic, we generated t-SNE visualization plots. Fig. 13(a) displays the distribution of raw traffic in the dimensionality-reduced feature space prior to clustering. The blue points (representing known malicious and benign samples from the training set) form well-defined and extremely compact clusters in the space. This is attributed to the compression of intra-class distances achieved by the Sieve contrastive learning module. The red points (representing the detected unknown traffic) are not mixed with known classes; instead, they form multiple independent, high-density clusters. This visually demonstrates that the features extracted by Sieve possess strong discriminative power, capable of naturally separating semantically distinct unknown attacks within the feature space.

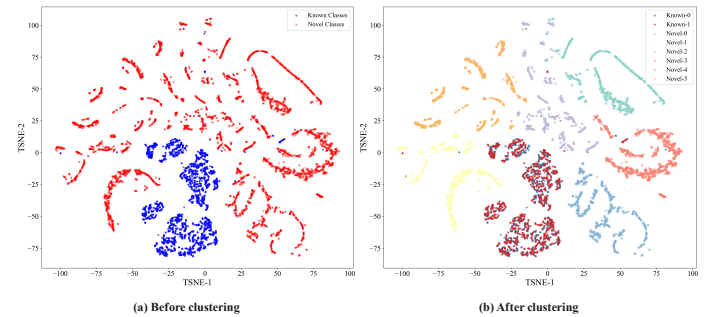


Fig. 13. t-SNE visualization of feature distributions on the IDS2018 Dataset: (a) Ground truth distribution of known and detected unknown traffic, and (b) Clustering results of unknown traffic.

Fig. 13(b) illustrates the clustering results after applying the estimated class number ($k = 6$), where different colors represent different clusters. It can be clearly observed that the unknown traffic is partitioned into 6 distinct clusters. Although

there are only 5 ground truth classes, causing one type of attack to be split into two clusters of similar colors (i.e., over-segmentation), distinct margins are still maintained between the clusters. These visualization results indicate that, despite the minor deviation in class number estimation, Sieve successfully decouples different attack behaviors within the semantic space. Security analysts need only review the centroid samples of these 6 clusters to quickly merge the two clusters belonging to the same attack type, thereby efficiently completing the labeling of new attacks.

E. Sensitivity Analysis

In this subsection, we perform sensitivity analysis on the critical hyperparameters in Sieve.

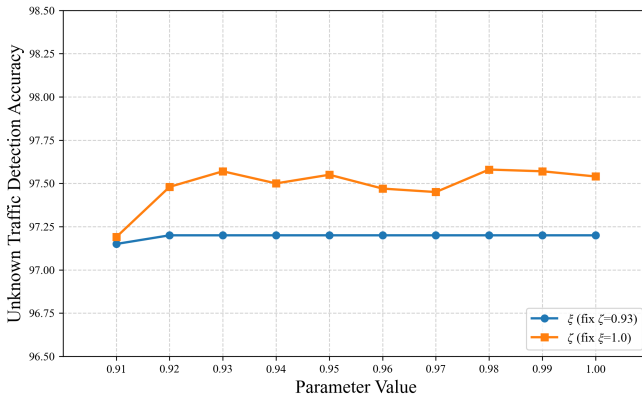


Fig. 14. Impact of hyperparameters ξ and ζ on Sieve performance.

(1) Effect of ξ and ζ .

Fig. 14 shows the impact of ξ and ζ on unknown traffic detection. Fixing $\xi = 1.0$ and varying ζ yields better performance than fixing ζ and varying ξ , indicating that the purity of the initially selected subset is more important than sample quantity. This high-purity subset provides reliable class centroids for subsequent expansion and detection.

The best performance is obtained when $\zeta = 0.93$, which balances sample recall and noise rejection. A lower ζ may admit ambiguous open-set noise, while a higher ζ may discard useful clean samples. In practice, when different classes exhibit strong feature overlap, ξ can be slightly relaxed while ζ should be increased accordingly to maintain the purity of the expanded subset.

(2) **Effect of K .** Fig. 15 illustrates the impact of neighbor count K on classification performance. While known class accuracy remains stable at approximately 95.5% across the testing range, unknown detection accuracy oscillates (97.1%–97.4%) for $K < 100$ before peaking at 97.6% at $K = 100$ and plateauing. This performance surge suggests that $K = 100$ optimally aligns with the feature space’s geometric density, providing a receptive field large enough to smooth noise and establish a robust consensus without inducing cross-category confusion. This alignment enables more precise intra-class compactness calculation, directly enhancing Mahalanobis distance-based unknown traffic detection.

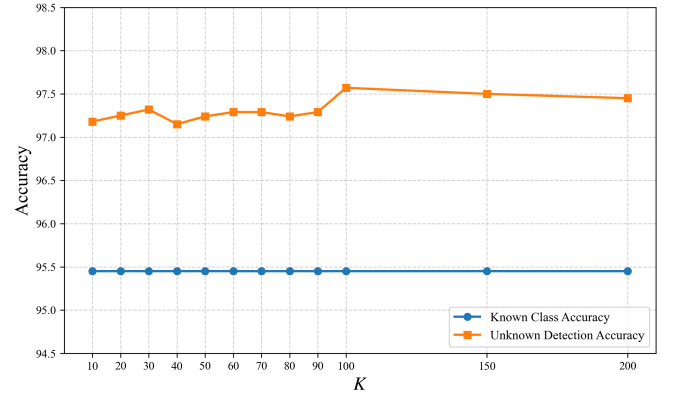


Fig. 15. Impact of hyperparameters K on Sieve performance.

TABLE IX
THE ABLATION STUDY OF KEY COMPONENTS UNDER DIFFERENT NOISE SCENARIOS.

Datasets	Scenarios	Method			
		w/o LC	w/o SE	w/o CL	Full
Mal_TLS2023	0.1Sym_0.5Open	93.94	95.18	94.72	95.45
	0.3Sym_0.5Open	93.38	94.89	94.78	95.40
	0.5Sym_0.5Open	90.91	94.56	93.88	94.96
	0.1Asym_0.5Open	93.29	94.88	93.82	94.85
	0.3Asym_0.5Open	91.99	94.20	94.03	95.20
	0.5Asym_0.5Open	89.36	94.18	94.67	95.25
DDoS2019	0.1Sym_0.5Open	94.74	98.06	99.14	98.48
	0.3Sym_0.5Open	89.94	98.08	97.90	98.51
	0.5Sym_0.5Open	86.77	98.06	97.12	98.58
	0.1Asym_0.5Open	95.75	98.10	98.97	99.12
	0.3Asym_0.5Open	88.87	98.02	98.22	99.07
	0.5Asym_0.5Open	86.82	98.09	97.87	99.05

F. Ablation Study

To verify the individual contribution of each component within the Sieve framework, we conduct a comprehensive ablation study. We evaluate four configurations: (1) Full, the complete Sieve framework; (2) w/o LC, which removes the noisy label correction module; (3) w/o SE, which excludes the subset expansion mechanism; and (4) w/o CL, which omits the self-supervised contrastive loss. These experiments are carried out across various noise ratios and types on the Mal_TLS2023 and DDoS2019 datasets to assess how each module influences the model’s classification accuracy.

The experimental results, summarized in Table IX, clearly demonstrate that every designed component is vital for maintaining robustness under mixed noise conditions. The Full model consistently achieves the highest accuracy across all scenarios, particularly in extreme cases like 0.5Asym_0.5Open. We observe that removing the label correction module results in the most significant performance collapse—dropping by over 12% in the DDoS2019 0.5Asym_0.5Open scenario—which underscores its foundational role in purifying the training set. Furthermore, the performance declines in the w/o SE and w/o CL variants indicate that subset expansion successfully recaptures valuable

samples, while contrastive loss is essential for learning the compact, noise-resistant feature representations necessary for precise detection.

VII. LIMITATIONS AND FUTURE WORKS

Over-Segmentation in Unknown-Traffic Labeling. The reliability of the unknown-traffic labeling module depends on the purity of the detection output. As observed in the experiments, imperfect detection can lead to over-segmentation (e.g., one true attack class being split into two clusters) and increases the need for manual cluster merging. To reduce this reliance on expert intervention, future work could explore: (1) automated post-processing heuristics that merge clusters based on centroid distance or distribution similarity metrics, and (2) semi-automated validation workflows that present only the most uncertain candidate merges to security experts, thereby focusing human effort on the most critical decisions.

Limitations of Synthetic Unknown-Traffic. This study evaluates unknown traffic by treating classes from external datasets as unknown. While this controlled strategy enables reproducible benchmarking, it has inherent limitations compared to real-world operations: (1) real unknown attacks may not form coherent, well-separated semantic clusters and may exhibit more scattered or anomalous feature patterns; (2) in practice, open-set noise samples are likely intertwined with closed-set noise in a far more complex manner than the clean batch-wise injection simulated here; and (3) the static protocol cannot assess model adaptation to gradual concept drift over time. Therefore, generalizing to the more chaotic, real-world unknown threats remains an important direction for future research.

VIII. CONCLUSION

In this paper, we propose the Sieve model, which effectively addresses the challenges of known traffic classification and unknown threat detection in scenarios with mixed label noise through a threefold collaborative mechanism. The Sieve framework consists of three closely cooperating modules: the noise label correction module utilizes neighborhood consistency metrics and prediction confidence analysis to accurately distinguish noisy samples and build a clean training set; the unknown traffic detection module employs Mahalanobis distance metrics to achieve accurate identification of unknown traffic in an optimized feature space; the traffic labeling module combines semi-supervised clustering with expert verification to support dynamic updates of the dataset. These three modules form a complete technical loop, ensuring the stability and adaptability of the framework in noisy environments. Comprehensive tests on four benchmark datasets demonstrate that Sieve maintains outstanding performance even under various noise conditions. Theoretical analysis further confirms that the framework effectively prevents information loss, eliminates label blind spots, and minimizes open set risks by optimizing the intra-class feature distribution to the greatest extent.

REFERENCES

- [1] M. Shen, J. Wu, K. Ye, K. Xu, G. Xiong, and L. Zhu, "Robust detection of malicious encrypted traffic via contrastive learning," *IEEE Transactions on Information Forensics and Security*, 2025.
- [2] Z. Zhao, Z. Li, Z. Song, W. Li, and F. Zhang, "Trident: A universal framework for fine-grained and class-incremental unknown traffic detection," in *Proceedings of the ACM Web Conference 2024*, 2024, pp. 1608–1619.
- [3] M. Shen, K. Ji, Z. Gao, Q. Li, L. Zhu, and K. Xu, "Subverting website fingerprinting defenses with robust traffic representation," in *32nd USENIX Security Symposium (USENIX Security 23)*, 2023, pp. 607–624.
- [4] S. Cui, C. Dong, M. Shen, Y. Liu, B. Jiang, and Z. Lu, "Cbseq: A channel-level behavior sequence for encrypted malware traffic detection," *IEEE Transactions on Information Forensics and Security*, vol. 18, pp. 5011–5025, 2023.
- [5] Q. Meng, Q. Yuan, J. Tao, P. Wang, S. Lu, G. Li, Y. Wang, and X. Guan, "When unknown threat meets label noise: A self-correcting framework," *IEEE Transactions on Dependable and Secure Computing*, 2025.
- [6] M. Kim and I. Lee, "Human-guided auto-labeling for network traffic data: The gelm approach," *Neural networks*, vol. 152, pp. 510–526, 2022.
- [7] J. L. Guerra, C. Catania, and E. Veas, "Datasets are not enough: Challenges in labeling network traffic," *Computers & Security*, vol. 120, p. 102810, 2022.
- [8] R. Sachdeva, F. R. Cordeiro, V. Belagiannis, I. Reid, and G. Carneiro, "Evidentialmix: Learning with combined open-set and closed-set noisy labels," in *Proceedings of the IEEE/CVF winter conference on applications of computer vision*, 2021, pp. 3607–3615.
- [9] B. Van Rooyen and R. C. Williamson, "A theory of learning with corrupted labels," *Journal of Machine Learning Research*, vol. 18, no. 228, pp. 1–50, 2018.
- [10] T. Dahanayaka, Y. Ginige, Y. Huang, G. Jourjon, and S. Seneviratne, "Robust open-set classification for encrypted traffic fingerprinting," *Computer Networks*, vol. 236, p. 109991, 2023.
- [11] Y. Chen, Z. Li, J. Shi, G. Gou, C. Liu, and G. Xiong, "Not afraid of the unseen: a siamese network based scheme for unknown traffic discovery," in *2020 IEEE Symposium on Computers and Communications (ISCC)*. IEEE, 2020, pp. 1–7.
- [12] J. Zhang, F. Li, F. Ye, and H. Wu, "Autonomous unknown-application filtering and labeling for dl-based traffic classifier update," in *IEEE INFOCOM 2020-IEEE Conference on Computer Communications*. IEEE, 2020, pp. 397–405.
- [13] M. Zhan, R. Zhao, X. Deng, Z. Xue, Q. Li, Z. Liu, G. Cheng, and K. Xu, "Flowrefiner: A robust traffic classification framework against label noise," in *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.
- [14] G. Li, R. Zhang, Z. Sun, L. Xing, and Y. Zhang, "Dsdir: A two-stage method for addressing noisy long-tailed problems in malicious traffic detection," in *ICASSP 2025-2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2025, pp. 1–5.
- [15] Q. Yuan, G. Gou, Y. Zhu, Y. Zhu, G. Xiong, and Y. Wang, "Mcre: A unified framework for handling malicious traffic with noise labels based on multidimensional constraint representation," *IEEE Transactions on Information Forensics and Security*, vol. 19, pp. 133–147, 2023.
- [16] Y. Qing, Q. Yin, X. Deng, Y. Chen, Z. Liu, K. Sun, K. Xu, J. Zhang, and Q. Li, "Low-quality training data only? a robust framework for detecting encrypted malicious network traffic," *arXiv preprint arXiv:2309.04798*, 2023.
- [17] X. Luo, C. Liu, G. Xiong, G. Gou, Z. Li, J. Shi, L. Guo, and B. Fang, "Baptism: A robust framework for encrypted malicious traffic identification with low-quality training data," *IEEE Transactions on Information Forensics and Security*, vol. 21, pp. 960–975, 2025.
- [18] Z. Zhao, Z. Li, X. Xie, J. Yu, F. Zhang, R. Zhang, B. Chen, X. Luo, M. Hu, and W. Ma, "Foss: Towards fine-grained unknown class detection against the open-set attack spectrum with variable legitimate traffic," *IEEE/ACM Transactions on Networking*, 2024.
- [19] X. Wang, Y. Wang, Y. Lai, Z. Hao, and A. X. Liu, "Reliable open-set network traffic classification," *IEEE Transactions on Information Forensics and Security*, 2025.
- [20] Q. Yang, W. He, M. Chen, H. Du, S. Shao, F. Wu, S. Liu, Y. Ji, and K. Ren, "End-to-end open-set semi-supervised learning for fine-grained encrypted traffic classification," *IEEE Transactions on Information Forensics and Security*, 2026.

- [21] Y. Ginige, B. Silva, T. Dahanayaka, and S. Seneviratne, "Trafficllm: Llm for improved open-set encrypted traffic analysis," *Available at SSRN 5074974*.
- [22] Y. Zhong, Z. Wang, X. Shi, J. Yang, and K. Li, "Rfg-helad: A robust fine-grained network traffic anomaly detection model based on heterogeneous ensemble learning," *IEEE Transactions on Information Forensics and Security*, 2024.
- [23] M. Salehi, H. Mirzaei, D. Hendrycks, Y. Li, M. H. Rohban, and M. Sabokrou, "A unified survey on anomaly, novelty, open-set, and out-of-distribution detection: Solutions and future challenges," *arXiv preprint arXiv:2110.14051*, 2021.
- [24] W. J. Scheirer, A. de Rezende Rocha, A. Sapkota, and T. E. Boulton, "Toward open set recognition," *IEEE transactions on pattern analysis and machine intelligence*, vol. 35, no. 7, pp. 1757–1772, 2012.
- [25] A. Bendale and T. E. Boulton, "Towards open set deep networks," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 1563–1572.
- [26] D. Barradas, N. Santos, L. Rodrigues, S. Signorello, F. M. Ramos, and A. Madeira, "Flowlens: Enabling efficient flow classification for ml-based network security applications," in *NDSS*, 2021.
- [27] J. Holland, P. Schmitt, N. Feamster, and P. Mittal, "New directions in automated traffic analysis," in *Proceedings of the 2021 ACM SIGSAC conference on computer and communications security*, 2021, pp. 3366–3383.
- [28] G. Zhou, Z. Liu, C. Fu, Q. Li, and K. Xu, "An efficient design of intelligent network data plane," in *32nd USENIX Security Symposium (USENIX Security 23)*, 2023, pp. 6203–6220.
- [29] C. Fu, Q. Li, and K. Xu, "Flow interaction graph analysis: Unknown encrypted malicious traffic detection," *IEEE/ACM Transactions on Networking*, vol. 32, no. 4, pp. 2972–2987, 2024.
- [30] Q. Meng, J. Tao, Q. Yuan, G. Li, Y. Wang, B. Gao, and S. Lu, "Detection of unknown attacks through encrypted traffic: A gaussian prototype-aided variational autoencoder framework," *IEEE Transactions on Information Forensics and Security*, 2025.
- [31] Q. Yuan, Q. Meng, J. Tao, G. Li, J. Fei, B. Lu, and Y. Wang, "Multi-agent for network security monitoring and warning: a generative ai solution," *IEEE Network*, 2025.
- [32] M. Shen, Y. Liu, L. Zhu, X. Du, and J. Hu, "Fine-grained webpage fingerprinting using only packet length information of encrypted traffic," *IEEE Transactions on Information Forensics and Security*, vol. 16, pp. 2046–2059, 2020.
- [33] S. Yang, X. Zheng, J. Li, J. Xu, and E. C. Ngai, "Cold: Collaborative label denoising framework for network intrusion detection," 2026.
- [34] X. Xia, B. Han, N. Wang, J. Deng, J. Li, Y. Mao, and T. Liu, "Extended t t: Learning with mixed closed-set and open-set noisy labels," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 3, pp. 3047–3058, 2022.
- [35] C. Fu, Q. Li, M. Shen, and K. Xu, "Realtime robust malicious traffic detection via frequency domain analysis," in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, 2021, pp. 3431–3446.
- [36] L. Gao, C. Fu, X. Deng, K. Xu, and Q. Li, "Wedjat: Detecting sophisticated evasion attacks via real-time causal analysis," in *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 1*, 2025, pp. 342–353.
- [37] C. Fu, Q. Li, E. Bertino, and K. Xu, "Training with only 1.0% samples: Malicious traffic detection via cross-modality feature fusion," in *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*, 2025, pp. 3930–3944.
- [38] M. Shen, J. Wu, J. Ai, Q. Li, C. Ren, K. Xu, and L. Zhu, "Swallow: a transfer-robust website fingerprinting attack via consistent feature learning," in *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*, 2025, pp. 1574–1588.
- [39] M. Shen, K. Ji, J. Wu, Q. Li, X. Kong, K. Xu, and L. Zhu, "Real-time website fingerprinting defense via traffic cluster anonymization," in *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2024, pp. 3238–3256.
- [40] M. Shen, J. Zhang, L. Zhu, K. Xu, and X. Du, "Accurate decentralized application identification via encrypted traffic analysis using graph neural networks," *IEEE Transactions on Information Forensics and Security*, vol. 16, pp. 2367–2380, 2021.
- [41] C. Feng, G. Tzimiropoulos, and I. Patras, "Ssr: An efficient and robust framework for learning with unknown label noise," *arXiv preprint arXiv:2111.11288*, 2021.
- [42] V. Schwag, M. Chiang, and P. Mittal, "Ssd: A unified framework for self-supervised outlier detection," *arXiv preprint arXiv:2103.12051*, 2021.
- [43] J. Yoder and C. E. Priebe, "Semi-supervised k-means++," *Journal of Statistical Computation and Simulation*, vol. 87, no. 13, pp. 2597–2608, 2017.
- [44] J. M. Johnson and T. M. Khoshgoftaar, "Survey on deep learning with class imbalance," *Journal of big data*, vol. 6, no. 1, pp. 1–54, 2019.
- [45] H. Zhang, M. Cisse, Y. N. Dauphin, and D. Lopez-Paz, "mixup: Beyond empirical risk minimization," *arXiv preprint arXiv:1710.09412*, 2017.
- [46] B. Liu, L. Zhan, Z. Lu, Y. Feng, L. Xue, and X.-M. Wu, "How good are llms at out-of-distribution detection?" *arXiv preprint arXiv:2308.10261*, 2023.
- [47] K. Han, S.-A. Rebuffi, S. Ehrhardt, A. Vedaldi, and A. Zisserman, "Autonovel: Automatically discovering and learning novel visual categories," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 10, pp. 6767–6781, 2021.
- [48] F. Gao, W. Zhong, Z. Cao, X. Peng, and Z. Li, "Opengcd: Assisting open world recognition with generalized category discovery," *arXiv preprint arXiv:2308.06926*, 2023.
- [49] S. Vaze, K. Han, A. Vedaldi, and A. Zisserman, "Generalized category discovery," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2022, pp. 7492–7501.
- [50] K. R. Gegenfurtner, "Praxis: Brent's algorithm for function minimization," *Behavior Research Methods, Instruments, & Computers*, vol. 24, no. 4, pp. 560–564, 1992.
- [51] D. L. Davies and D. W. Bouldin, "A cluster separation measure," *IEEE transactions on pattern analysis and machine intelligence*, no. 2, pp. 224–227, 2009.
- [52] Q. Yuan, C. Liu, W. Yu, Y. Zhu, G. Xiong, Y. Wang, and G. Gou, "Boau: Malicious traffic detection with noise labels based on boundary augmentation," *Computers & Security*, vol. 131, p. 103300, 2023.
- [53] N. Wickramasinghe, A. Shaghagh, G. Tsudik, and S. Jha, "Sok: Decoding the enigma of encrypted network traffic classifiers," in *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2025, pp. 1825–1843.
- [54] I. Sharafaldin, A. H. Lashkari, S. Hakak, and A. A. Ghorbani, "Developing realistic distributed denial of service (ddos) attack dataset and taxonomy," in *2019 international carmahan conference on security technology (ICCSST)*. IEEE, 2019, pp. 1–8.
- [55] I. Sharafaldin, A. H. Lashkari, A. A. Ghorbani *et al.*, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," *ICISSP*, vol. 1, no. 2018, pp. 108–116, 2018.
- [56] K. Lee, K. Lee, H. Lee, and J. Shin, "A simple unified framework for detecting out-of-distribution samples and adversarial attacks," *Advances in neural information processing systems*, vol. 31, 2018.
- [57] W. Liu, X. Wang, J. Owens, and Y. Li, "Energy-based out-of-distribution detection," *Advances in neural information processing systems*, vol. 33, pp. 21 464–21 475, 2020.
- [58] W. Zhou, F. Liu, and M. Chen, "Contrastive out-of-distribution detection for pretrained transformers," *arXiv preprint arXiv:2104.08812*, 2021.
- [59] C. Fu, Q. Li, M. Shen, and K. Xu, "Detecting tunneled flooding traffic via deep semantic analysis of packet length patterns," in *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, 2024, pp. 3659–3673.
- [60] C. Fu, Q. Li, K. Xu, and J. Wu, "Point cloud analysis for ml-based malicious traffic detection: Reducing majorities of false positive alarms," in *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, 2023, pp. 1005–1019.
- [61] Y. Yan, Q. Yuan, W. Niu, X. Wang, Y. Zhu, and Y. Wang, "Fcal: An asynchronous federated contrastive semi-supervised learning approach for network traffic classification," in *International Conference on Information and Communications Security*. Springer, 2025, pp. 419–437.
- [62] K. Chen, Q. Yuan, Q. Meng, D. Li, Y. Zhu, G. Yu, X. Wang, and Y. Wang, "Robust intrusion detection in cps: A pre-training-based multi-view feature collaboration and correlation analysis method," *Computer Networks*, p. 112129, 2026.